

Resumo de Aprendizado de Máquina

joao.vmcardsoso

11 de junho de 2026

Conteúdo

1	Métodos de Ensemble Learning	7
1.1	O Problema Fundamental: Por que um Modelo Isolado é Insuficiente?	8
1.1.1	O Dilema do Especialista	8
1.1.2	O Paralelo no Aprendizado de Máquina	8
1.1.3	A Solução dos Ensembles	8
1.2	Intuição Geométrica: Onde os Ensembles se Destacam	8
1.2.1	A Analogia do Mapa	8
1.2.2	Por que isso funciona matematicamente?	9
1.3	Vocabulário Essencial e Mapas Conceituais	9
1.3.1	Termos que Você Precisa Saber	9
1.3.2	Mapa Conceitual das Relações	9
1.3.3	Quando Usar Cada Método?	9
1.4	Exemplo Didático Passo-a-Passo	9
1.4.1	Problema: Classificar Flores (Íris) com 3 Classificadores Fracos	9
1.5	Aprofundamento: Bagging (Bootstrap Aggregation)	11
1.5.1	Definição Formal	11
1.5.2	Por que o Bootstrap Reduz a Correlação?	11
1.5.3	Out-of-Bag (OOB) Error	11
1.6	Aprofundamento: Boosting e AdaBoost	11
1.6.1	AdaBoost: Algoritmo Completo	11
1.6.2	Interpretação Estatística do AdaBoost	12
1.6.3	Por que $\alpha_m = \frac{1}{2} \ln((1 - \epsilon_m)/\epsilon_m)$?	12
1.7	Mistura de Especialistas (Mixtures of Experts)	12
1.7.1	Arquitetura Probabilística	12
1.7.2	Treinamento via EM	12
1.7.3	Conexão com Redes Neurais Modernas	13
1.8	Análise Formal de Bias e Variância em Ensembles	14
1.8.1	Decomposição Bias-Variância para Regressão (Bishop)	14
1.8.2	Efeito do Bagging sobre Bias e Variância	14
1.8.3	Efeito do Boosting sobre Bias e Variância	14
1.9	Comparação Entre Métodos: Tabela Abrangente	14
1.10	Limitações e Quando os Ensembles Falham	15
1.10.1	Custos e Desvantagens	15
1.10.2	Quando o Ensemble NÃO Ajuda	15
1.11	Pesquisa Atual e Fronteiras	15
1.11.1	<i>Mixture of Experts</i> (MoE) em LLMs	15
1.11.2	<i>Knowledge Distillation</i>	15
1.11.3	<i>Dynamic Ensemble Selection</i> (DES)	15
1.11.4	Aprendizado Contínuo e Esquecimento Catastrófico	15
1.12	Resumo e Conexões com Outros Capítulos	15
1.12.1	O que Você Deve Lembrar	15
1.12.2	Conexões com Outros Capítulos	16
1.12.3	<i>Cheat Sheet</i> para Revisão Rápida (Leitores Experientes)	16

2	Aprendizado Não-Supervisionado	17
2.1	O Problema Fundamental: O que Fazer sem Rótulos?	18
2.1.1	A Analogia do Arqueólogo	18
2.1.2	O Contraste com o Supervisionado	18
2.1.3	As Três Tarefas Fundamentais	18
2.1.4	Exemplo Didático: O Problema dos Vinhos	18
2.2	Intuição Geométrica: O Espaço dos Dados	18
2.2.1	A Analogia do Arquipélago	18
2.2.2	Por que Reduzir Dimensionalidade?	19
2.3	Vocabulário Essencial e Mapas Conceituais	19
2.3.1	Termos que Você Precisa Saber	19
2.3.2	Mapa Conceitual das Relações	19
2.3.3	Quando Usar Cada Método?	19
2.4	Exemplo Didático Passo-a-Passo	19
2.4.1	Problema: Agrupar Pontos no Plano com K-means	19
2.5	Aprofundamento: K-means e Algoritmos de Clustering	21
2.5.1	K-means: Definição Formal	21
2.5.2	Complexidade e Inicialização	21
2.5.3	Agrupamento Hierárquico	21
2.6	Aprofundamento: Estimativa de Densidade Não-Paramétrica	22
2.6.1	Janelas de Parzen (Kernel Density Estimation)	22
2.6.2	Escolha da Largura de Banda h	22
2.6.3	Método dos K-Vizinhos Mais Próximos (KNN) para Densidade	22
2.7	Aprofundamento: Redução de Dimensionalidade	22
2.7.1	Análise de Componentes Principais (PCA)	22
2.7.2	Interpretação dos Autovalores	23
2.7.3	Análise de Componentes Independentes (ICA)	23
2.7.4	Técnicas Modernas: UMAP e t-SNE	23
2.8	Análise Formal: Propriedades dos Algoritmos	24
2.8.1	Convergência do K-means	24
2.8.2	Comparação entre Métodos de Redução de Dimensionalidade	24
2.8.3	Limitações do K-means	24
2.9	Comparação Entre Métodos de Clustering	24
2.10	Limitações e Quando o Não-Supervisionado Falha	25
2.10.1	Custos e Desvantagens	25
2.10.2	Quando o Não-Supervisionado NÃO Ajuda	25
2.11	Pesquisa Atual e Fronteiras	25
2.11.1	Aprendizado Auto-Supervisionado (<i>Self-Supervised Learning</i>)	25
2.11.2	Adaptação de Domínio Não-Supervisionada	25
2.11.3	Clustering em Larga Escala	25
2.11.4	Deep Clustering	25
2.12	Resumo e Conexões com Outros Capítulos	26
2.12.1	O que Você Deve Lembrar	26
2.12.2	Conexões com Outros Capítulos	26
2.12.3	<i>Cheat Sheet</i> para Revisão Rápida (Leitores Experientes)	26
3	Aprendizado Semi-Supervisionado	27
3.1	O Problema Fundamental: Como Aproveitar Dados Não Rotulados?	28
3.1.1	A Analogia do Professor e do Estágio	28
3.1.2	O Contraste com Outros Paradigmas	28
3.1.3	A Motivação Econômica e Prática	28
3.1.4	Exemplo Didático: O Problema dos Dígitos	28
3.2	Intuição Geométrica: A Hipótese do Cluster	29
3.2.1	A Hipótese Fundamental do SSL	29
3.2.2	Por que isso funciona matematicamente?	29
3.2.3	A Analogia do Mapa Topográfico	29
3.3	Vocabulário Essencial e Mapas Conceituais	29
3.3.1	Termos que Você Precisa Saber	29

3.3.2	Mapa Conceitual das Relações	30
3.3.3	Quando Usar SSL?	30
3.4	Exemplo Didático Passo-a-Passo	30
3.4.1	Problema: Classificação com Poucos Rótulos	30
3.5	Aprofundamento: Modelos Probabilísticos e EM	31
3.5.1	O Algoritmo EM para SSL	31
3.5.2	EM com Dados Parcialmente Rotulados	31
3.5.3	Identificabilidade em Misturas	31
3.6	Aprofundamento: Auto-Treinamento (Self-Training)	31
3.6.1	Algoritmo Genérico	31
3.6.2	Métricas de Confiança	31
3.6.3	Silver Datasets e Augmented SBERT	32
3.7	Aprofundamento: Aprendizado Ativo (Active Learning)	32
3.7.1	Definição e Motivação	32
3.7.2	Estratégias de Seleção	32
3.7.3	Aprendizado Ativo vs. SSL Passivo	32
3.8	Análise Formal: Condições para o Sucesso do SSL	33
3.8.1	Quando o SSL Ajuda?	33
3.8.2	Quando o SSL Pode Ser Prejudicial?	33
3.8.3	Identificabilidade: A Base Teórica	33
3.9	Comparação Entre Métodos SSL	33
3.10	Limitações e Quando o SSL Falha	33
3.10.1	Custos e Desvantagens	33
3.10.2	Quando o SSL NÃO Ajuda	34
3.11	Pesquisa Atual e Fronteiras	34
3.11.1	Dilema Estabilidade-Plasticidade	34
3.11.2	Redução de Alucinações via RAG	34
3.11.3	Hibridização Indutiva-Analítica	34
3.11.4	SSL em LLMs e Auto-Supervisão	34
3.11.5	Deep Semi-Supervised Learning	34
3.12	Resumo e Conexões com Outros Capítulos	34
3.12.1	O que Você Deve Lembrar	34
3.12.2	Conexões com Outros Capítulos	35
3.12.3	<i>Cheat Sheet</i> para Revisão Rápida (Leitores Experientes)	35
4	Aprendizado por Transferência	36
4.1	O Problema Fundamental: Como Aprender com Poucos Dados?	37
4.1.1	A Analogia do Poliglota	37
4.1.2	O Contraste com o Paradigma Tradicional	37
4.1.3	A Motivação Econômica e Prática	37
4.1.4	Exemplo Didático: Reconhecimento de Cachorros	37
4.2	Intuição Geométrica: A Hierarquia de Características	38
4.2.1	A Analogia da Pirâmide de Conhecimento	38
4.2.2	Por que Transferir Funciona?	38
4.3	Vocabulário Essencial e Mapas Conceituais	38
4.3.1	Termos que Você Precisa Saber	38
4.3.2	Mapa Conceitual das Relações	39
4.3.3	Quando Usar Transferência?	39
4.4	Exemplo Didático Passo-a-Passo: Fine-Tuning de um Classificador de Imagens	39
4.4.1	Problema: Classificar Raças de Cachorros	39
4.4.2	Passo a Passo	39
4.5	Aprofundamento: Fine-Tuning e Congelamento de Camadas	41
4.5.1	Fine-Tuning: Formulação Matemática	41
4.5.2	Congelamento de Camadas	41
4.5.3	Taxa de Aprendizado Diferenciada	41
4.6	Aprofundamento: PEFT e LoRA	41
4.6.1	O Problema do Fine-Tuning Completo	41
4.6.2	LoRA (Low-Rank Adaptation)	41

4.6.3	Vantagens do LoRA	42
4.7	Aprofundamento: Destilação de Conhecimento (Knowledge Distillation)	42
4.7.1	Definição Formal	42
4.7.2	Temperatura na Destilação	42
4.8	Análise Formal: Condições para Transferência Bem-Sucedida	43
4.8.1	Similaridade entre Tarefas	43
4.8.2	Teoria de Generalização	43
4.9	Comparação Entre Métodos de Transferência	43
4.10	Limitações e Quando a Transferência Falha	43
4.10.1	Custos e Desvantagens	43
4.10.2	Quando a Transferência NÃO Ajuda	44
4.11	Pesquisa Atual e Fronteiras	44
4.11.1	Modelos de Fundação e Escalabilidade	44
4.11.2	Aprendizado ao Longo da Vida (Lifelong Learning)	44
4.11.3	Adaptação de Domínio Não-Supervisionada	44
4.11.4	Destilação de Conhecimento para LLMs	44
4.11.5	Hibridização Indutiva-Analítica	44
4.12	Resumo e Conexões com Outros Capítulos	44
4.12.1	O que Você Deve Lembrar	44
4.12.2	Conexões com Outros Capítulos	45
4.12.3	<i>Cheat Sheet</i> para Revisão Rápida (Leitores Experientes)	45
5	Otimização de Modelos de Machine Learning	46
5.1	O Problema Fundamental: Como Encontrar o Mínimo?	47
5.1.1	A Analogia do Andarilho Cego na Montanha	47
5.1.2	O Paralelo com Aprendizado	47
5.1.3	O Contraste Entre Otimização e Generalização	47
5.1.4	Exemplo Didático: Descida do Gradiente em 1D	47
5.2	Intuição Geométrica: A Paisagem de Erro	48
5.2.1	Superfícies Convexas vs. Não-Convexas	48
5.2.2	O Problema dos Platôs e Vales Estreitos	48
5.2.3	A Analogia da Bola em uma Tábua de Madeira	48
5.3	Vocabulário Essencial e Mapas Conceituais	48
5.3.1	Termos que Você Precisa Saber	48
5.3.2	Mapa Conceitual das Relações	49
5.3.3	Quando Usar Cada Método?	49
5.4	Exemplo Didático Passo-a-Passo: Classificação com SGD	49
5.4.1	Problema: Regressão Logística em 2D	49
5.5	Aprofundamento: Métodos de Primeira Ordem	51
5.5.1	Gradiente Descendente: Versões	51
5.5.2	Momentum e Nesterov Accelerated Gradient	51
5.5.3	Otimizadores Modernos: Adam e AdamW	51
5.6	Aprofundamento: Métodos de Segunda Ordem	51
5.6.1	Método de Newton	51
5.6.2	Vantagens e Desvantagens	52
5.6.3	Gradiente Conjugado	52
5.7	Aprofundamento: Regularização e Restrições	52
5.7.1	Regularização L2 (Ridge)	52
5.7.2	Regularização L1 (Lasso)	52
5.7.3	Multiplicadores de Lagrange	52
5.8	Análise Formal: Convergência e Taxas de Aprendizado	53
5.8.1	Convergência para Funções Convexas	53
5.8.2	Convergência para Funções Não-Convexas	53
5.8.3	Escolha da Taxa de Aprendizado	53
5.9	Comparação Entre Otimizadores	53
5.10	Limitações e Quando a Otimização Falha	53
5.10.1	Custos e Desvantagens	53
5.10.2	Quando a Otimização Não Ajuda	54

5.11	Pesquisa Atual e Fronteiras	54
5.11.1	Otimização para LLMs	54
5.11.2	Otimização para PEFT (LoRA)	54
5.11.3	Otimização Neuronal e Meta-Aprendizado	54
5.11.4	Otimização Distribuída	54
5.11.5	Otimização Quântica	54
5.12	Resumo e Conexões com Outros Capítulos	54
5.12.1	O que Você Deve Lembrar	54
5.12.2	Conexões com Outros Capítulos	55
5.12.3	<i>Cheat Sheet</i> para Revisão Rápida (Leitores Experientes)	55

Referências

- **Bishop:** BISHOP, Christopher M. *Pattern Recognition and Machine Learning*. Springer, 2006.
- **Raschka:** RASCHKA, Sebastian. *Build a Large Language Model (From Scratch)*. Manning, 2024.
- **Alammar:** ALAMMAR, Jay; GROOTENDORST, Maarten. *Hands-On Large Language Models*. O'Reilly, 2024.
- **Sorvisto:** SORVISTO, Dayne. *MLOps Lifecycle Toolkit*. Apress, 2023.
- **Duda:** DUDA, Richard O.; HART, Peter E.; STORK, David G. *Pattern Classification*. Wiley, 2001.
- **Mitchell:** MITCHELL, Tom M. *Machine Learning*. McGraw Hill, 1997.
- **Hastie:** HASTIE, Trevor; TIBSHIRANI, Robert; FRIEDMAN, Jerome. *The Elements of Statistical Learning*. Springer, 2009.
- **Goodfellow:** GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. *Deep Learning*. MIT Press, 2016.

Capítulo 1

Métodos de Ensemble Learning

Visão Geral do Capítulo

Este capítulo apresenta os métodos de Ensemble Learning (aprendizado por comitês ou ensembles). A ideia central é simples, mas poderosa: **combinar múltiplos modelos pode produzir um resultado melhor do que qualquer modelo individual**. O capítulo está organizado para atender tanto leitores que buscam uma primeira exposição ao tema quanto aqueles que desejam uma revisão técnica aprofundada.

- **Seções 1 a 4:** Para todos os leitores. Apresentam intuição, motivação, analogias e exemplos didáticos.
- **Seções 5 a 7:** Aprofundamento técnico. Contêm equações, algoritmos e derivações.
- **Seções 8 a 10:** Discussão avançada. Incluem análises formais, comparações entre métodos e pesquisa atual.
- **Seção 11:** Resumo e conexões com outros capítulos.

1.1 O Problema Fundamental: Por que um Modelo Isolado é Insuficiente?

1.1.1 O Dilema do Especialista

Imagine que você precisa de um diagnóstico médico importante. Você consulta **um único médico**. Ele pode ser excelente, mas também pode, por exemplo: estar cansado naquele dia, ter viés baseado em experiências anteriores, faltar conhecimento em uma subárea específica ou interpretar um exame de forma diferente.

Agora imagine que você consulta **cinco médicos especialistas** em diferentes áreas e combina suas opiniões. O diagnóstico provavelmente será mais robusto. Isso é *Ensemble Learning*.

1.1.2 O Paralelo no Aprendizado de Máquina

Um modelo de machine learning isolado sofre de dois tipos principais de erros (Bishop; Hastie):

- **Variância alta:** O modelo é muito sensível a pequenas mudanças nos dados de treinamento. Imagine um dardo que acerta regiões diferentes a cada lançamento, mas a média é o alvo. É preciso **muitos lançamentos** para acertar.
- **Bias alto:** O modelo faz suposições tão fortes que sistematicamente erra. Imagine um dardo que sempre acerta o mesmo lugar, mas que está longe do alvo. Não adianta lançar mais vezes.

Um único modelo, especialmente árvores de decisão profundas, tende a ter **bias baixo mas variância alta** (sobreajuste). Modelos muito simples, como regressão linear, tendem a ter **variância baixa mas bias alto** (subajuste).

1.1.3 A Solução dos Ensembles

“Dois pesos não fazem uma balança, mas muitos pesos fazem um ensemble.”

Os métodos de ensemble atacam esses dois problemas de formas diferentes:

Método	Reduz principalmente	Como funciona
Bagging	Variância	Treina modelos em paralelo, cada um em uma amostra diferente dos dados
Boosting	Bias	Treina modelos em sequência, cada um corrigindo os erros do anterior
Stacking	Ambos	Treina um meta-modelo que aprende a melhor forma de combinar os outros

Tabela 1.1: Principais métodos de ensemble e seus efeitos.

Exemplo didático (classificação binária):

Suponha que você tem 5 classificadores independentes, cada um com acurácia de 60% (apenas ligeiramente melhor que o acaso). Se eles erram de forma **independente**, a chance da maioria votar errado é:

$$P(\text{erro da maioria}) = \sum_{k=3}^5 \binom{5}{k} (0.4)^k (0.6)^{5-k} \approx 0.317$$

Ou seja, o ensemble de 5 classificadores fracos atinge **68,3% de acurácia** — uma melhoria significativa! (Bishop; Duda)

1.2 Intuição Geométrica: Onde os Ensembles se Destacam

1.2.1 A Analogia do Mapa

Imagine que você está desenhando uma fronteira de decisão entre duas classes em um plano:

- **Uma árvore de decisão profunda** é como um mapa desenhado por alguém que recorta cada detalhe de uma cidade — segue perfeitamente os dados de treinamento (baixo bias), mas, se você muda uma casa de lugar, o mapa muda completamente (alta variância).

- **Uma regressão logística** é como um mapa que usa uma linha reta — não captura detalhes (alto bias), mas é robusto a pequenas mudanças (baixa variância).

Um ensemble de árvores (Bagging) é como pedir para 100 pessoas desenharem o mapa (cada uma vendo uma amostra ligeiramente diferente da cidade) e depois **tirar a média** dos mapas. Detalhes idiossincráticos se cancelam; as características verdadeiramente importantes emergem.

1.2.2 Por que isso funciona matematicamente?

Sejam $y_m(x)$ as previsões de M modelos, $f(x)$ a função verdadeira. O erro quadrático médio do ensemble é:

$$E \left[\left(\frac{1}{M} \sum_m y_m(x) - f(x) \right)^2 \right] = \underbrace{\left(\frac{1}{M} \sum_m E[y_m(x) - f(x)] \right)^2}_{(\text{Bias do ensemble})^2} + \underbrace{\frac{1}{M^2} \sum_m \sum_n \text{Cov}(y_m, y_n)}_{\text{Variância do ensemble}}$$

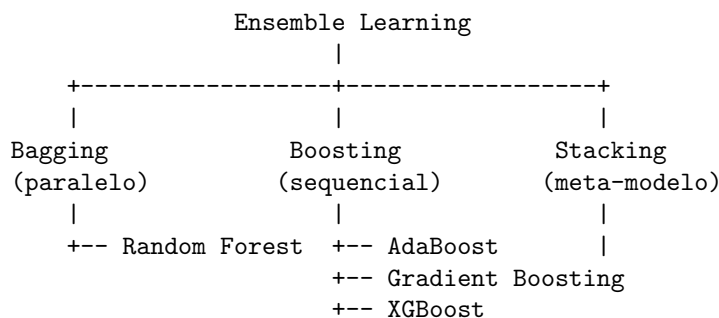
Se os erros são **não correlacionados** ($\text{Cov} \approx 0$), a variância cai por um fator de M ; se são **perfeitamente correlacionados**, a variância não se reduz. O segredo do Bagging é justamente **quebrar a correlação** entre os modelos via bootstrap (Bishop; Hastie).

1.3 Vocabulário Essencial e Mapas Conceituais

1.3.1 Termos que Você Precisa Saber

- **Ensemble / Comitê:** Conjunto de modelos combinados.
- **Weak learner:** classificador fraco (acurácia > 50%, porém baixa). O AdaBoost exige isso.
- **Bootstrap:** amostragem com reposição do conjunto de dados.
- **Bagging:** Bootstrap + Aggregating.
- **Boosting:** treinamento sequencial com reponderação dos dados.
- **Gating network:** rede que decide qual especialista usar em *Mixture of Experts*.
- **Diversidade:** medida de quão diferentes são os modelos no ensemble. Mais diversidade geralmente melhora o ensemble.

1.3.2 Mapa Conceitual das Relações



1.3.3 Quando Usar Cada Método?

1.4 Exemplo Didático Passo-a-Passo

1.4.1 Problema: Classificar Flores (Íris) com 3 Classificadores Fracos

Vamos criar um ensemble simples para entender o mecanismo por trás desses métodos.

Dados (exemplo simplificado): - 6 amostras, 2 classes (Setosa = +1, Versicolor = -1) - Características: comprimento da pétala (x_1) e largura da sépala (x_2)

Amostra:	1	2	3	4	5	6
x1:	1.2	1.4	1.5	4.5	4.8	5.0
x2:	3.0	3.2	2.9	2.8	3.0	2.7
Classe:	+1	+1	+1	-1	-1	-1

Situação	Método recomendado	Razão
Poucos dados, modelo instável (ex: árvore profunda)	Bagging	Reduz variância
Modelo muito simples (ex: stump de decisão)	Boosting	Reduz bias iterativamente
Tempo de treinamento limitado	Bagging (paralelizável)	Modelos treinam independentemente
Dados muito grandes	Boosting (XGBoost é otimizado)	Cada iteração foca nos erros
Interpretabilidade necessária	Bagging + árvores rasas	Florestas permitem análise de importância

Tabela 1.2: Orientação para escolha do método de ensemble.

Classificadores fracos (stumps de decisão — árvores de 1 nó):

$h_1(x)$: “Se $x_1 < 3.0$, prediz +1; senão -1”. - Verificando: amostras 1, 2, 3 ($x_1 < 3$) $\rightarrow +1$ (correto); amostras 4, 5, 6 ($x_1 \geq 3$) $\rightarrow -1$ (correto). Este classificador já é perfeito — mas, para fins didáticos, vamos fingir que ele é fraco.

Para criar um classificador verdadeiramente fraco, usaremos um limiar que não separa perfeitamente. Suponha que $h_1(x)$ erre 2 das 6 amostras (erro $\epsilon = 2/6 \approx 0,33$). O AdaBoost então:

1. Atribui pesos iguais $w_n = 1/6$.
2. Treina h_1 , calcula $\epsilon_1 = 0,33$, $\alpha_1 = \ln((1 - 0,33)/0,33) = \ln(2) \approx 0,69$.
3. Aumenta o peso das amostras erradas.
4. Treina h_2 focado nessas amostras.
5. Combinação final: $Y = \text{sign}(0,69 \cdot h_1(x) + \alpha_2 \cdot h_2(x) + \dots)$.

A intuição: cada novo classificador é forçado a aprender o que o anterior não aprendeu.

1.5 Aprofundamento: Bagging (Bootstrap Aggregation)

1.5.1 Definição Formal

O Bagging gera M conjuntos de dados via *bootstrap*: cada conjunto D_m é formado por N amostras sorteadas **com reposição** do conjunto original D de tamanho N . Aproximadamente 63,2% das amostras originais aparecem em cada D_m (as demais são repetições) (Bishop; Hastie).

Para cada D_m , treina-se um modelo $y_m(x)$. A predição final é:

$$y_{\text{bagging}}(x) = \frac{1}{M} \sum_{m=1}^M y_m(x) \quad (\text{regressão})$$
$$y_{\text{bagging}}(x) = \arg \max_k \sum_{m=1}^M \mathbf{1}[y_m(x) = k] \quad (\text{classificação - votação majoritária})$$

1.5.2 Por que o Bootstrap Reduz a Correlação?

Se os modelos fossem treinados no mesmo conjunto D , eles seriam idênticos (correlação = 1). O bootstrap introduz pequenas variações nos dados de treinamento, gerando, assim, modelos diferentes. A correlação entre dois modelos bootstrap é aproximadamente:

$$\rho \approx \frac{\text{Var}_{\text{bootstrap}}[y]}{\text{Var}_{\text{original}}[y]}$$

Quanto mais **instável** o modelo (ex: árvore profunda), menor a correlação e maior o ganho do Bagging (Bishop; Duda).

1.5.3 Out-of-Bag (OOB) Error

Para cada amostra x_n , aproximadamente 36,8% dos conjuntos bootstrap **não** a contêm. Essas são as amostras *out-of-bag*. Podemos usá-las para validar o modelo **sem precisar de um conjunto de validação separado**:

$$\text{OOB erro} = \frac{1}{N} \sum_{n=1}^N \mathbf{1}[y_{\text{ensemble, sem } n}(x_n) \neq t_n]$$

Onde $y_{\text{ensemble, sem } n}$ é a predição usando apenas os modelos que não viram x_n no treinamento (Bishop; Hastie).

1.6 Aprofundamento: Boosting e AdaBoost

1.6.1 AdaBoost: Algoritmo Completo

Dados: $\{(x_n, t_n)\}_{n=1}^N, t_n \in \{-1, +1\}$

Inicialize pesos: $w_n^{(1)} = \frac{1}{N}$ para $n = 1, \dots, N$

Para $m = 1$ até M :

1. Treine classificador $y_m(x)$ minimizando o erro ponderado:

$$J_m = \sum_{n=1}^N w_n^{(m)} \mathbf{1}[y_m(x_n) \neq t_n]$$

2. Calcule a taxa de erro ponderada:

$$\epsilon_m = \frac{\sum_{n=1}^N w_n^{(m)} \mathbf{1}[y_m(x_n) \neq t_n]}{\sum_{n=1}^N w_n^{(m)}}$$

3. Calcule o coeficiente de importância do classificador:

$$\alpha_m = \frac{1}{2} \ln \left(\frac{1 - \epsilon_m}{\epsilon_m} \right)$$

4. Atualize os pesos:

$$w_n^{(m+1)} = w_n^{(m)} \exp(-\alpha_m t_n y_m(x_n))$$

Nota: se $y_m(x_n) = t_n$ (acerto), o peso é multiplicado por $e^{-\alpha_m} < 1$ (diminui); se $y_m(x_n) \neq t_n$ (erro), o peso é multiplicado por $e^{+\alpha_m} > 1$ (aumenta).

5. Normalize os pesos: $w_n^{(m+1)} \leftarrow \frac{w_n^{(m+1)}}{\sum_j w_j^{(m+1)}}$

Predição final:

$$Y_M(x) = \text{sign} \left(\sum_{m=1}^M \alpha_m y_m(x) \right)$$

(Fonte: Bishop; Duda)

1.6.2 Interpretação Estatística do AdaBoost

O AdaBoost minimiza sequencialmente uma função de erro exponencial (Bishop):

$$E = \sum_{n=1}^N \exp(-t_n f(x_n)), \quad \text{onde } f(x) = \sum_m \alpha_m y_m(x)$$

Esta é uma aproximação convexa da função 0-1 (erro de classificação). A minimização ocorre via *forward stagewise additive modeling* (Hastie).

1.6.3 Por que $\alpha_m = \frac{1}{2} \ln((1 - \epsilon_m)/\epsilon_m)$?

Se o classificador m tem erro ϵ_m , a melhor combinação linear α satisfaz (derivação):

Derivando a função de erro exponencial em relação a α , obtém-se:

$$\frac{\partial E}{\partial \alpha} = 0 \Rightarrow e^{2\alpha} = \frac{1 - \epsilon}{\epsilon} \Rightarrow \alpha = \frac{1}{2} \ln \frac{1 - \epsilon}{\epsilon}$$

Intuição: Quanto menor ϵ_m , maior α_m ; ou seja, classificadores melhores têm mais peso na decisão final (Duda; Bishop).

1.7 Mistura de Especialistas (Mixtures of Experts)

1.7.1 Arquitetura Probabilística

A *Mixture of Experts* (MoE) assume que diferentes regiões do espaço de entrada são melhor modeladas por diferentes especialistas (Bishop). O modelo é:

$$p(t|x) = \sum_{k=1}^K \pi_k(x) p(t|x, k)$$

Onde: - $p(t|x, k)$ é a predição do especialista k (pode ser uma regressão linear, rede neural, etc.) - $\pi_k(x)$ é a saída da *gating network*, com $\sum_k \pi_k(x) = 1$, $\pi_k(x) \geq 0$

1.7.2 Treinamento via EM

O algoritmo *Expectation-Maximization* (EM) é usado (Bishop):

Passo E: Calcular a responsabilidade do especialista k pela amostra n :

$$r_{nk} = \frac{\pi_k(x_n) p(t_n|x_n, k)}{\sum_{j=1}^K \pi_j(x_n) p(t_n|x_n, j)}$$

Passo M: Atualizar: - Especialistas: maximizar $\sum_n r_{nk} \ln p(t_n|x_n, k)$ (cada especialista é treinado ponderado por suas responsabilidades) - *Gating network*: maximizar $\sum_n \sum_k r_{nk} \ln \pi_k(x_n)$

1.7.3 Conexão com Redes Neurais Modernas

As arquiteturas *Mixture of Experts* (MoE) em *Large Language Models* (ex: Mixtral 8x7B) são a implementação em escala massiva deste conceito (Raschka; Alammr). A diferença prática é que cada “especialista” é uma camada *feedforward* dentro de um *transformer*, e a *gating network* é esparsa (ativa apenas 2 dos K especialistas por *token*).

1.8 Análise Formal de Bias e Variância em Ensembles

1.8.1 Decomposição Bias-Variância para Regressão (Bishop)

Para um modelo de regressão com perda quadrática, o erro esperado se decompõe como:

$$\mathbb{E}[(y(x) - f(x))^2] = \underbrace{(\mathbb{E}[y(x)] - f(x))^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}[(y(x) - \mathbb{E}[y(x)])^2]}_{\text{Variância}} + \underbrace{\sigma^2}_{\text{Ruído irreduzível}}$$

1.8.2 Efeito do Bagging sobre Bias e Variância

O Bagging produz $y_{\text{bag}}(x) = \mathbb{E}_{\mathcal{D}}[y_{\mathcal{D}}(x)]$ no limite $M \rightarrow \infty$ (onde $\mathcal{D}$ é a distribuição bootstrap). Então:

$$\text{Bias}_{\text{bag}} = \mathbb{E}[y_{\text{bag}}(x)] - f(x) = \mathbb{E}_{\mathcal{D}}[\mathbb{E}[y_{\mathcal{D}}(x)]] - f(x) = \text{Bias}_{\text{original}}$$

O bias **não se altera**. A variância, porém, é reduzida:

$$\text{Var}_{\text{bag}} = \mathbb{E}_{\mathcal{D}}[\text{Var}[y_{\mathcal{D}}(x)|\mathcal{D}]] \leq \text{Var}_{\text{original}}$$

Em modelos lineares estáveis, a redução de variância pode ser pequena. Em árvores de decisão (alta variância), a redução é drástica (Bishop; Hastie).

1.8.3 Efeito do Boosting sobre Bias e Variância

O Boosting foca em reduzir o bias sequencialmente. Cada novo modelo é treinado para corrigir os erros dos anteriores. No entanto, se executado por muitas iterações, o Boosting pode **aumentar a variância** (sobreajuste). Este é o *trade-off* fundamental (Duda; Hastie).

Método	Efeito no Bias	Efeito na Variância
Bagging	Inalterado	Reduz
Boosting (poucas iterações)	Reduz	Pouco afetada
Boosting (muitas iterações)	Reduz (muito)	Pode aumentar
Stacking	Pode reduzir ambos	Dependente do meta-modelo

Tabela 1.3: Efeitos dos métodos de ensemble sobre bias e variância.

1.9 Comparação Entre Métodos: Tabela Abrangente

Característica	Bagging	Boosting (AdaBoost)	Stacking
Treinamento	Paralelo	Sequencial	Paralelo + Meta
Modelos base	Qualquer, mas melhor com instáveis	<i>Weak learners</i> (erro $\leq 0,5$)	Qualquer
Ponderação dos dados	Uniforme (bootstrap)	Adaptativa (erros pesam mais)	Uniforme
Combinação final	Média / votação	Média ponderada (α_m)	Meta-modelo treinado
Suscetibilidade a <i>overfitting</i>	Baixa	Média (controlada por M)	Média
Paralelizável	Sim (completamente)	Limitada (sequencial)	Sim (especialistas)
Interpretabilidade	Média (<i>Random Forest</i> ajuda)	Baixa	Baixa
Complexidade $O(\cdot)$	$M \cdot T_{\text{treino}}$	$M \cdot T_{\text{treino}}$	$K \cdot T_{\text{treino}} + T_{\text{meta}}$

Tabela 1.4: Comparação entre os principais métodos de ensemble. Fonte: Bishop; Duda; Hastie

1.10 Limitações e Quando os Ensembles Falham

1.10.1 Custos e Desvantagens

1. **Custo computacional:** Treinar M modelos é M vezes mais caro.
2. **Memória:** Armazenar M modelos pode ser inviável para implantação em dispositivos de borda.
3. **Interpretabilidade reduzida:** Um ensemble de 100 árvores é muito mais difícil de explicar do que uma regressão logística.
4. *Diminishing returns:* Após certo M , o ganho marginal se torna pequeno.
5. **Risco de *overfitting* em Boosting:** Se M é muito grande, o ensemble pode se ajustar excessivamente ao ruído (Mitchell; Duda).

1.10.2 Quando o Ensemble NÃO Ajuda

- **Modelos base já são muito bons e estáveis:** Se um único modelo tem baixo bias e baixa variância, o ensemble adiciona pouco.
- **Modelos base são idênticos:** Se não há diversidade, não há ganho.
- **Dados são extremamente escassos:** O *bootstrap* de um *dataset* pequeno gera conjuntos muito similares, reduzindo o benefício do Bagging.
- **Os modelos base são todos ruins na mesma região:** Se todos os classificadores fracos erram as mesmas amostras, o Boosting não terá o que corrigir.

1.11 Pesquisa Atual e Fronteiras

1.11.1 *Mixture of Experts* (MoE) em LLMs

Arquiteturas modernas como Mixtral 8x7B e GPT-4 (especula-se) utilizam MoE para escalar o número de parâmetros sem aumentar o custo computacional proporcionalmente (Raschka; Alammari). A inovação é ativar apenas 2 dos K especialistas por *token* (esparsidade), reduzindo o custo de inferência.

1.11.2 *Knowledge Distillation*

Uma alternativa ao uso de ensembles em produção é a destilação (Hinton et al., 2015): treina-se um modelo pequeno (“aluno”) para imitar as saídas do ensemble (“professor”). O resultado é um modelo único que aproxima o desempenho do ensemble com fração do custo (Sorvisto).

1.11.3 *Dynamic Ensemble Selection* (DES)

Pesquisas recentes buscam selecionar dinamicamente, para cada nova entrada x , quais modelos do ensemble são mais confiáveis naquela região do espaço (Bishop; Mitchell). Isso contrasta com a combinação fixa (mesmos pesos para todo x).

1.11.4 Aprendizado Contínuo e Esquecimento Catastrófico

Quando um ensemble é usado em ambientes não estacionários (dados mudam ao longo do tempo), surge o dilema estabilidade-plasticidade: como adicionar novos especialistas para aprender novos padrões sem que os antigos “esqueçam” o que sabiam? (Mitchell; Goodfellow)

1.12 Resumo e Conexões com Outros Capítulos

1.12.1 O que Você Deve Lembrar

1. **Ideia central:** Combinar modelos supera o modelo isolado quando eles erram de forma diferente.
2. **Bagging:** Reduz variância. Treinamento paralelo. Ideal para modelos instáveis (árvores).
3. **Boosting:** Reduz bias. Treinamento sequencial. Ideal para modelos fracos.
4. **Votação ou média:** forma mais simples de combinar (funciona bem quando os modelos são igualmente bons).
5. **Stacking/MoE:** combinação aprendida (meta-modelo ou *gating network*). Mais flexível, porém exige mais dados.

1.12.2 Conexões com Outros Capítulos

Este capítulo sobre Ensemble Learning se conecta com os demais capítulos do resumo das seguintes formas:

- **Aprendizado Não-Supervisionado (Capítulo 2):** O Bagging utiliza o método de *bootstrap*, que é uma técnica de reamostragem baseada em conceitos não-supervisionados. Além disso, a *Mixture of Experts* (MoE) discutida neste capítulo é treinada via algoritmo EM, que também aparece no contexto de modelos de mistura Gaussianas no Capítulo 2.
- **Aprendizado Semi-Supervisionado (Capítulo 3):** O conceito de *co-training* (não abordado em detalhe neste capítulo) é um ensemble em que cada modelo é treinado em uma visão diferente dos dados, usando pseudo-rótulos gerados por uma visão para ensinar a outra — uma ponte direta entre ensemble e SSL.
- **Aprendizado por Transferência (Capítulo 4):** A destilação de conhecimento (*knowledge distillation*) é uma forma de transferir o “saber” de um ensemble (professor) para um modelo menor (aluno), combinando os dois paradigmas. O Capítulo 4 aborda destilação em detalhe.
- **Otimização de Modelos (Capítulo 5):** O AdaBoost minimiza sequencialmente uma função de erro exponencial; entender os métodos de otimização apresentados no Capítulo 5 ajuda a compreender por que e como o AdaBoost converge.

Resumo das relações:

Capítulo	Conexão com Ensemble
Não-Supervisionado (2)	Bootstrap, algoritmo EM
Semi-Supervisionado (3)	Co-training (ensemble de visões)
Transferência (4)	Destilação de conhecimento (professor → aluno)
Otimização (5)	AdaBoost como otimização exponencial

1.12.3 Cheat Sheet para Revisão Rápida (Leitores Experientes)

Quero...	Uso...	Fórmula chave
Reduzir variância	Bagging	$y_{\text{bag}} = \frac{1}{M} \sum y_m$
Reduzir bias com modelos fracos	Boosting	$\alpha_m = \frac{1}{2} \ln \frac{1-\epsilon_m}{\epsilon_m}$
Adaptação local	MoE	$p(t x) = \sum \pi_k(x) p(t x, k)$
Limite teórico de erro	<i>Weighted Majority</i>	$\frac{\text{erro total}}{\text{erro do melhor}} \leq \frac{\ln(1/\beta)}{1-\beta} \quad .$

Tabela 1.5: Resumo rápido para consulta.

Capítulo 2

Aprendizado Não-Supervisionado

Visão Geral do Capítulo

Este capítulo apresenta o Aprendizado Não-Supervisionado (*Unsupervised Learning*), um paradigma do aprendizado de máquina em que os dados não possuem rótulos ou supervisão externa. O objetivo é descobrir estruturas intrínsecas, padrões ocultos ou representações úteis a partir apenas das características dos dados.

O capítulo está organizado para atender tanto leitores que buscam uma primeira exposição ao tema quanto aqueles que desejam uma revisão técnica aprofundada.

- **Seções 1 a 4:** Para todos os leitores. Apresentam intuição, motivação, analogias e exemplos didáticos.
- **Seções 5 a 8:** Aprofundamento técnico. Contêm equações, algoritmos e derivações.
- **Seções 9 a 11:** Discussão avançada. Incluem análises formais, comparações entre métodos e pesquisa atual.
- **Seção 12:** Resumo e conexões com outros capítulos.

2.1 O Problema Fundamental: O que Fazer sem Rótulos?

2.1.1 A Analogia do Arqueólogo

Imagine que você é um arqueólogo e descobre uma caverna cheia de objetos nunca antes vistos. Não há nenhum manual, nenhuma etiqueta, nenhum especialista para dizer o que cada objeto é. O que você faz?

Você começa a observar: alguns objetos são redondos e pesados; outros são alongados e leves; alguns estão sempre juntos; outros aparecem isolados. Sem saber os nomes, você **agrupa** objetos similares, **identifica padrões** e talvez até **descubra relações** entre eles.

Isso é aprendizado não-supervisionado: extrair conhecimento a partir de dados brutos, sem rótulos prévios (Bishop; Duda).

2.1.2 O Contraste com o Supervisionado

Aspecto	Supervisionado	Não-Supervisionado
Dados de treino	(x_n, t_n) (entrada + rótulo)	(x_n) (apenas entrada)
Objetivo	Prever rótulos de novas amostras	Descobrir estrutura nos dados
Exemplo	Classificar e-mails como “spam” ou “não spam”	Agrupar e-mails por assunto
Supervisão	“Professor” fornece respostas	Sem “professor”

Tabela 2.1: Diferenças entre aprendizado supervisionado e não-supervisionado.

2.1.3 As Três Tarefas Fundamentais

O aprendizado não-supervisionado se organiza em torno de três grandes famílias de problemas (Bishop):

1. **Clustering (agrupamento):** Dividir os dados em grupos onde elementos do mesmo grupo são mais similares entre si do que com elementos de outros grupos.
2. **Estimativa de densidade:** Modelar a distribuição de probabilidade que gerou os dados.
3. **Redução de dimensionalidade:** Encontrar representações de menor dimensão que preservam a estrutura essencial dos dados.

2.1.4 Exemplo Didático: O Problema dos Vinhos

Suponha que você tem 100 amostras de vinho, cada uma descrita por 13 características (teor alcoólico, acidez, etc.), mas você **não sabe** a que vinícola cada amostra pertence. O que você pode fazer?

- **Clustering:** Agrupar as amostras em (digamos) 3 grupos. Depois, você pode descobrir que esses grupos correspondem, na verdade, a 3 vinícolas diferentes — mesmo sem nunca ter visto os rótulos (Duda).
- **Redução de dimensionalidade:** Projetar as 13 características em 2 dimensões para visualizar os dados em um gráfico, identificando possíveis separações entre grupos (Bishop).
- **Estimativa de densidade:** Determinar se uma nova amostra de vinho é “típica” ou “anômala” em relação às 100 amostras existentes.

2.2 Intuição Geométrica: O Espaço dos Dados

2.2.1 A Analogia do Arquipélago

Imagine que cada ponto de dado é uma ilha em um vasto oceano. As ilhas próximas têm características semelhantes (clima, vegetação, relevo). As ilhas distantes são diferentes.

- **Clustering** é como traçar fronteiras no oceano para definir arquipélagos: ilhas próximas pertencem ao mesmo grupo.

- **Redução de dimensionalidade** é como criar um mapa bidimensional do arquipélago, perdendo alguns detalhes (como a altura exata de cada montanha), mas preservando as distâncias relativas entre as ilhas.
- **Estimativa de densidade** é como medir, para cada região do oceano, quão “lotada” ela está de ilhas.

2.2.2 Por que Reduzir Dimensionalidade?

O *curse of dimensionality* (maldição da dimensionalidade) é um fenômeno crucial: em espaços de alta dimensão, as distâncias entre pontos tendem a se tornar uniformes, e o volume do espaço cresce exponencialmente (Bishop). Com d dimensões, para manter a mesma densidade de pontos, precisamos de um número de amostras que cresce como $N \propto k^d$, o que rapidamente se torna inviável.

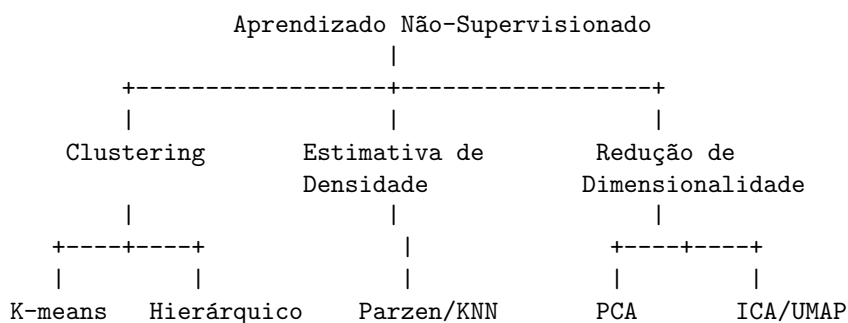
A redução de dimensionalidade combate esse problema: projeta os dados em um espaço de menor dimensão ($d' \ll d$) onde os métodos tradicionais funcionam melhor.

2.3 Vocabulário Essencial e Mapas Conceituais

2.3.1 Termos que Você Precisa Saber

- **Cluster**: grupo de pontos com alta similaridade interna e baixa similaridade externa.
- **Centróide**: ponto central de um cluster (geralmente a média dos pontos).
- **Dendrograma**: diagrama em árvore que mostra o histórico de fusões no clustering hierárquico.
- **Métrica de distância**: função que mede a dissimilaridade entre dois pontos (ex: Euclidiana, Manhattan, cosseno).
- **Elbow method**: técnica para escolher o número K de clusters no K-means observando a “curva” da inércia.
- **Kernel**: função usada em estimativa de densidade (ex: Gaussiano, Epanechnikov).
- **Manifold (variedade)**: superfície de baixa dimensão onde os dados de alta dimensão realmente “vivem”.

2.3.2 Mapa Conceitual das Relações



2.3.3 Quando Usar Cada Método?

2.4 Exemplo Didático Passo-a-Passo

2.4.1 Problema: Agrupar Pontos no Plano com K-means

Vamos usar o K-means em um exemplo simples para entender o mecanismo.

Dados (6 pontos em 2D):

Ponto:	A	B	C	D	E	F
x:	1	2	3	8	9	10
y:	2	2	2	8	8	8

Visualmente, há dois grupos: $\{A, B, C\}$ no canto inferior esquerdo e $\{D, E, F\}$ no canto superior direito.

Objetivo	Método recomendado	Razão
Agrupar dados com formato esférico	K-means	Rápido, simples, escalável
Agrupar sem saber K	Hierárquico	Dendrograma permite explorar K
Agrupar formas complexas	DBSCAN (ver seção de pesquisa)	Não assume esfericidade
Visualizar dados em $2D/3D$	PCA (linear) ou UMAP (não-linear)	Preserva estrutura local/global
Estimar densidade suavemente	Janelas de Parzen	Não-paramétrico, flexível
Separar fontes independentes	ICA	Ideal para sinais misturados

Tabela 2.2: Orientação para escolha do método não-supervisionado.

Passo 1: Escolher $K = 2$ e inicializar centróides aleatórios. Suponha: - Centróide 1: $C_1 = (1.5, 2.5)$ (próximo a A e B) - Centróide 2: $C_2 = (8.5, 7.5)$ (próximo a E e F)

Passo 2 (Atribuição): Calcular a distância de cada ponto a cada centróide (distância Euclidiana: $\sqrt{(x_p - x_c)^2 + (y_p - y_c)^2}$).

Para ponto $A = (1, 2)$: - Distância a C_1 : $\sqrt{(1 - 1.5)^2 + (2 - 2.5)^2} = \sqrt{0.25 + 0.25} = \sqrt{0.5} \approx 0.71$ - Distância a C_2 : $\sqrt{(1 - 8.5)^2 + (2 - 7.5)^2} = \sqrt{56.25 + 30.25} = \sqrt{86.5} \approx 9.30$ - A é atribuído ao cluster 1.

Para ponto $D = (8, 8)$: - Distância a C_1 : $\sqrt{(8 - 1.5)^2 + (8 - 2.5)^2} = \sqrt{42.25 + 30.25} = \sqrt{72.5} \approx 8.51$ - Distância a C_2 : $\sqrt{(8 - 8.5)^2 + (8 - 7.5)^2} = \sqrt{0.25 + 0.25} = \sqrt{0.5} \approx 0.71$ - D é atribuído ao cluster 2.

O processo continua para todos os pontos. Provavelmente, $\{A, B, C\}$ vão para o cluster 1, e $\{D, E, F\}$ vão para o cluster 2.

Passo 3 (Atualização): Recalcular centróides como a média dos pontos em cada cluster.

Cluster 1: média de $\{(1, 2), (2, 2), (3, 2)\} = ((1 + 2 + 3)/3, (2 + 2 + 2)/3) = (6/3, 6/3) = (2, 2)$

Cluster 2: média de $\{(8, 8), (9, 8), (10, 8)\} = ((8 + 9 + 10)/3, (8 + 8 + 8)/3) = (27/3, 24/3) = (9, 8)$

Passo 4: Repetir atribuição com os novos centróides $(2, 2)$ e $(9, 8)$. Os pontos não mudam de cluster (convergência). Fim.

O K-means encontrou corretamente os dois grupos!

2.5 Aprofundamento: K-means e Algoritmos de Clustering

2.5.1 K-means: Definição Formal

O K-means particiona N observações em K clusters ($K \leq N$), minimizando a soma dos quadrados das distâncias intra-cluster (Bishop). Seja $S = \{S_1, S_2, \dots, S_K\}$ a partição. O objetivo é:

$$\arg \min_S \sum_{k=1}^K \sum_{x \in S_k} \|x - \mu_k\|^2$$

Onde $\mu_k = \frac{1}{|S_k|} \sum_{x \in S_k} x$ é o centróide do cluster S_k .

O algoritmo alterna entre dois passos:

1. **Atribuição:** Cada ponto é atribuído ao cluster cujo centróide está mais próximo:

$$S_k^{(t)} = \{x_p : \|x_p - \mu_k^{(t)}\|^2 \leq \|x_p - \mu_j^{(t)}\|^2 \forall j\}$$

2. **Atualização:** Os centróides são recalculados como a média dos pontos no cluster:

$$\mu_k^{(t+1)} = \frac{1}{|S_k^{(t)}|} \sum_{x \in S_k^{(t)}} x$$

O algoritmo converge (não necessariamente para o ótimo global) porque a inércia total $\sum_k \sum_{x \in S_k} \|x - \mu_k\|^2$ decresce monotonicamente (Bishop; Duda).

2.5.2 Complexidade e Inicialização

A complexidade do K-means é $O(N \cdot K \cdot d \cdot T)$, onde: - N : número de amostras - K : número de clusters - d : dimensionalidade - T : número de iterações (geralmente pequeno, $T \ll N$)

A inicialização dos centróides influencia o resultado. Métodos comuns: - **Random:** escolher K pontos aleatórios como centróides iniciais. - **K-means++:** escolher o primeiro centróide aleatoriamente e os demais com probabilidade proporcional à distância ao centróide mais próximo (melhora a convergência) (Bishop; Hastie).

2.5.3 Agrupamento Hierárquico

Diferente do K-means, o clustering hierárquico não exige o número de clusters K *a priori*. O método **aglomerativo** (bottom-up) começa com cada ponto em seu próprio cluster e funde os pares mais similares iterativamente (Duda). As medidas de similaridade mais comuns são:

- **Single linkage (ligação simples):** distância mínima entre elementos dos dois clusters:

$$d_{\text{single}}(A, B) = \min_{x \in A, y \in B} \|x - y\|$$

- **Complete linkage (ligação completa):** distância máxima entre elementos:

$$d_{\text{complete}}(A, B) = \max_{x \in A, y \in B} \|x - y\|$$

- **Average linkage (ligação média):** distância média entre todos os pares:

$$d_{\text{avg}}(A, B) = \frac{1}{|A||B|} \sum_{x \in A} \sum_{y \in B} \|x - y\|$$

O resultado é visualizado em um **dendrograma**, uma árvore onde a altura de cada fusão indica a dissimilaridade naquele ponto (Duda; Bishop).

2.6 Aprofundamento: Estimativa de Densidade Não-Paramétrica

2.6.1 Janelas de Parzen (Kernel Density Estimation)

Quando a forma da distribuição é desconhecida, métodos não-paramétricos não assumem uma família paramétrica (ex: Gaussiana). A estimativa de densidade por *kernel* (Janelas de Parzen) é dada por (Bishop):

$$p(x) = \frac{1}{N} \sum_{n=1}^N \frac{1}{h^d} K\left(\frac{x - x_n}{h}\right)$$

Onde: - $K(\cdot)$ é uma função *kernel* (geralmente Gaussiana, mas podendo ser retangular, Epanechnikov, etc.) - h é o parâmetro de largura de banda (*bandwidth*) - d é a dimensionalidade

O kernel Gaussiano é:

$$K(u) = \frac{1}{(2\pi)^{d/2}} \exp\left(-\frac{\|u\|^2}{2}\right)$$

2.6.2 Escolha da Largura de Banda h

O parâmetro h controla o suavizamento (Bishop; Duda): - h pequeno: estimativa com alta variância (muitos “picos”espúrios) - h grande: estimativa com alto bias (oversmoothing)

Uma regra prática comum é a *Silverman’s rule of thumb*:

$$h = \left(\frac{4\hat{\sigma}^5}{3N}\right)^{1/5} \approx 1.06\hat{\sigma}N^{-1/5}$$

onde $\hat{\sigma}$ é o desvio padrão estimado dos dados (para o caso unidimensional Gaussiano).

2.6.3 Método dos K-Vizinhos Mais Próximos (KNN) para Densidade

Diferentemente do Parzen (que fixa h e deixa K variar), o método KNN fixa K e ajusta o volume (Bishop). Para um ponto x , encontre a esfera de raio $R_K(x)$ que contém exatamente K pontos. A densidade estimada é:

$$p(x) = \frac{K}{N \cdot V_d \cdot R_K(x)^d}$$

Onde V_d é o volume da esfera unitária em d dimensões: $V_d = \frac{\pi^{d/2}}{\Gamma(d/2+1)}$.

Intuição: em regiões densas, $R_K(x)$ é pequeno; em regiões esparsas, $R_K(x)$ é grande.

2.7 Aprofundamento: Redução de Dimensionalidade

2.7.1 Análise de Componentes Principais (PCA)

A PCA busca a projeção linear dos dados em um subespaço de dimensão $d' < d$ que maximiza a variância retida (ou, equivalentemente, minimiza o erro de reconstrução quadrático) (Bishop).

Sejam X a matriz de dados ($N \times d$), centrada (média zero). A matriz de covariância é:

$$\Sigma = \frac{1}{N} X^T X$$

Os autovetores de Σ são as **componentes principais**, ordenados pelos autovalores decrescentes $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d$. A projeção nos primeiros d' autovetores:

$$Z = X \cdot W_{d'}$$

Onde $W_{d'}$ é a matriz ($d \times d'$) contendo os primeiros d' autovetores.

2.7.2 Interpretação dos Autovalores

O autovalor λ_i representa a variância retida pela i -ésima componente principal. A fração da variância total retida pelas primeiras d' componentes é:

$$\text{Variância retida} = \frac{\sum_{i=1}^{d'} \lambda_i}{\sum_{j=1}^d \lambda_j}$$

Essa métrica é usada para escolher d' (ex: manter 95% da variância total) (Bishop).

2.7.3 Análise de Componentes Independentes (ICA)

Diferente da PCA, que busca componentes **não correlacionados**, a ICA busca componentes **estatisticamente independentes**, sendo ideal para separação cega de fontes (Duda). O modelo é:

$$x = As$$

Onde x são os dados observados (mistura), s são as fontes independentes (desconhecidas), e A é a matriz de mistura. O objetivo é estimar $W = A^{-1}$ tal que $s = Wx$ tenha componentes maximamente independentes (medidas por curtose ou negentropia).

2.7.4 Técnicas Modernas: UMAP e t-SNE

Para visualização em 2D ou 3D de dados de alta dimensão, métodos não-lineares como **t-SNE** e **UMAP** são preferidos à PCA (Alammar).

O **UMAP** (*Uniform Manifold Approximation and Projection*) constrói um grafo fuzzy de similaridades no espaço de alta dimensão e busca uma incorporação de baixa dimensão que preserve a topologia desse grafo (Alammar). Sua principal vantagem sobre t-SNE é a escalabilidade e a preservação tanto da estrutura local quanto da global.

2.8 Análise Formal: Propriedades dos Algoritmos

2.8.1 Convergência do K-means

O K-means é um caso particular do algoritmo *Expectation-Maximization* (EM) para um modelo de mistura Gaussiana onde: - A covariância é fixa como $\sigma^2 I$ (esférica) - $\sigma^2 \rightarrow 0$ (limite de componentes rígidos)

A função de perda (inércia) decresce monotonicamente, garantindo convergência a um **mínimo local**. Diferentes inicializações podem levar a diferentes mínimos locais (Bishop; Duda).

2.8.2 Comparação entre Métodos de Redução de Dimensionalidade

Método	Linear/Não-Linear	Preserva	Aplicação típica
PCA	Linear	Variância global	Visualização rápida, pré-processamento
ICA	Linear	Independência	Separação de fontes (sinais)
t-SNE	Não-linear	Estrutura local	Visualização de clusters
UMAP	Não-linear	Local + global	Visualização de embeddings de texto
Autoencoder (não-linear)	Não-linear (rede neural)	Reconstrução	Aprendizado de representações

Tabela 2.3: Comparação entre métodos de redução de dimensionalidade. Fonte: Bishop; Alammari

2.8.3 Limitações do K-means

O K-means tem limitações importantes (Bishop; Duda):

1. **Assume clusters esféricos:** funciona mal quando os clusters têm formas alongadas ou irregulares.
2. **Sensível a outliers:** um ponto muito distante pode deslocar o centróide significativamente.
3. **Requer K especificado *a priori*:** o *elbow method* é heurístico e nem sempre claro.
4. **Convergência a mínimo local:** diferentes inicializações produzem resultados diferentes.

Alternativas como **DBSCAN** (baseado em densidade) ou **Gaussian Mixture Models** (clusters elípticos) superam algumas dessas limitações.

2.9 Comparação Entre Métodos de Clustering

Característica	K-means	Hierárquico	DBSCAN	GMM
Forma dos clusters	Esférica	Qualquer (com linkage adequado)	Qualquer	Elíptica
Número K	Especificado	Não necessário (dendrograma)	Não necessário	Especificado
Outliers	Afetam	Afetam	Identifica	Afetam
Escalabilidade	Alta ($O(NKdT)$)	Baixa ($O(N^3)$)	Média ($O(N \log N)$)	Média
Probabilístico	Não	Não	Não	Sim

Tabela 2.4: Comparação entre algoritmos de clustering. Fonte: Bishop; Duda; Hastie

2.10 Limitações e Quando o Não-Supervisionado Falha

2.10.1 Custos e Desvantagens

1. **Falta de avaliação objetiva:** sem rótulos, como saber se o clustering está “correto”? Métricas como silhueta e Davies-Bouldin ajudam, mas são heurísticas.
2. **Maldição da dimensionalidade:** em alta dimensão, todas as distâncias se tornam semelhantes, e o clustering perde significado.
3. **Resultados arbitrários:** diferentes algoritmos (ou diferentes inicializações) podem produzir resultados radicalmente diferentes.
4. **Dificuldade de interpretação:** os clusters encontrados podem não corresponder a conceitos significativos para o domínio.

2.10.2 Quando o Não-Supervisionado NÃO Ajuda

- **Dados completamente aleatórios:** se não há estrutura, qualquer clustering encontrará grupos espúrios.
- **Dados de alta dimensão com poucas amostras:** a maldição da dimensionalidade torna as distâncias não informativas.
- **Quando se precisa de previsões para novos pontos:** muitas técnicas (como o dendrograma hierárquico) não produzem diretamente uma função de predição.

2.11 Pesquisa Atual e Fronteiras

2.11.1 Aprendizado Auto-Supervisionado (*Self-Supervised Learning*)

O auto-supervisionado é uma evolução recente do não-supervisionado que gera rótulos a partir da própria estrutura dos dados (Raschka). Exemplos incluem:

- **Predição de palavra mascarada (BERT):** o modelo aprende a prever palavras omitidas em uma frase.
- **Predição da próxima palavra (GPT):** o modelo aprende a sequência de tokens.
- **Contrastive learning (SimCLR):** o modelo aprende representações aproximando aumentos da mesma imagem e afastando imagens diferentes.

O auto-supervisionado alcança desempenho competitivo com supervisionado em muitas tarefas, sem usar rótulos humanos (Raschka; Alammari).

2.11.2 Adaptação de Domínio Não-Supervisionada

Técnicas como **TSDAE** (*Transformer-based Sequential Denoising Auto-Encoder*) adaptam modelos de *embeddings* a domínios específicos (ex: textos jurídicos) usando apenas dados não rotulados do novo domínio (Alammari). Isso reduz drasticamente a necessidade de dados anotados.

2.11.3 Clustering em Larga Escala

Com o crescimento dos *datasets* (milhões ou bilhões de pontos), algoritmos como **Faiss** (meta da Facebook) e **HDBSCAN** (versão hierárquica do DBSCAN) permitem clustering escalável. Pesquisas atuais focam em métodos *one-pass* (que processam dados em fluxo) e *sublinear* (que não exigem todas as distâncias aos pares).

2.11.4 Deep Clustering

A integração de redes neurais profundas com clustering (ex: Deep Embedded Clustering) aprende representações e clusters simultaneamente, superando métodos tradicionais em dados complexos como imagens (Goodfellow).

2.12 Resumo e Conexões com Outros Capítulos

2.12.1 O que Você Deve Lembrar

1. **Ideia central:** extrair estrutura de dados não rotulados.
2. **K-means:** iterativo, minimiza distâncias intra-cluster, rápido, esférico.
3. **Hierárquico:** gera dendrograma, não exige K , custo $O(N^3)$.
4. **PCA:** projeção linear que maximiza variância, pré-processamento padrão.
5. **UMAP/t-SNE:** visualização não-linear de dados de alta dimensão.
6. **Estimativa de densidade:** Parzen (fixa h) ou KNN (fixa K).

2.12.2 Conexões com Outros Capítulos

Este capítulo sobre Aprendizado Não-Supervisionado se conecta com os demais capítulos do resumo das seguintes formas:

- **Ensemble Learning (Capítulo 1):** O Bagging utiliza *bootstrap* (reamostragem), que é um conceito fundamentado em estatística não-supervisionada. O K-means discutido neste capítulo é um caso especial do algoritmo EM, que também aparece na *Mixture of Experts* do Capítulo 1.
- **Aprendizado Semi-Supervisionado (Capítulo 3):** O SSL é construído diretamente sobre as ferramentas do não-supervisionado: a hipótese do cluster (*cluster assumption*) é fundamental para SSL, e algoritmos de clustering (como EM) são usados para propagar rótulos de pontos rotulados para não rotulados.
- **Aprendizado por Transferência (Capítulo 4):** O pré-treinamento auto-supervisionado (*self-supervised learning*) — mencionado na Seção 10 deste capítulo — é uma forma de aprendizado não-supervisionado em larga escala que serve como base para modelos de fundação discutidos no Capítulo 4 (ex: BERT, GPT).
- **Otimização de Modelos (Capítulo 5):** O K-means é um caso particular do algoritmo *Expectation-Maximization* (EM), que é um método de otimização para variáveis latentes. O Capítulo 5 aborda otimização de primeira e segunda ordem, que são base para o funcionamento do EM.

Resumo das relações:

Capítulo	Conexão com Não-Supervisionado
Ensemble (1)	Bootstrap, EM na Mixture of Experts
Semi-Supervisionado (3)	Hipótese do cluster, propagação de rótulos
Transferência (4)	Auto-supervisionado como pré-treinamento
Otimização (5)	EM como algoritmo de otimização

2.12.3 Cheat Sheet para Revisão Rápida (Leitores Experientes)

Quero...	Uso...	Fórmula chave
Agrupar dados esféricos	K-means	$\min_S \sum_{k=1}^K \sum_{x \in S_k} \ x - \mu_k\ ^2$
Agrupar sem saber K	Hierárquico	Dendrograma
Visualizar dados 2D	PCA	$Z = X \cdot W_{d'}$
Visualizar dados 2D (não-linear)	UMAP / t-SNE	Preserva topologia local
Estimar densidade	Parzen	$p(x) = \frac{1}{N} \sum_{n=1}^N \frac{1}{h^d} K\left(\frac{x - x_n}{h}\right)$
Separar fontes independentes	ICA	$x = As, s = Wx$

Tabela 2.5: Resumo rápido para consulta.

Capítulo 3

Aprendizado Semi-Supervisionado

Visão Geral do Capítulo

Este capítulo apresenta o Aprendizado Semi-Supervisionado (*Semi-Supervised Learning* — SSL), um paradigma que se situa entre o aprendizado supervisionado e o não-supervisionado. A ideia central é aproveitar uma pequena quantidade de dados rotulados juntamente com um grande volume de dados não rotulados para construir modelos melhores do que os que usariam apenas os dados rotulados.

O capítulo está organizado para atender tanto leitores que buscam uma primeira exposição ao tema quanto aqueles que desejam uma revisão técnica aprofundada.

- **Seções 1 a 4:** Para todos os leitores. Apresentam intuição, motivação, analogias e exemplos didáticos.
- **Seções 5 a 7:** Aprofundamento técnico. Contêm equações, algoritmos e derivações.
- **Seções 8 a 10:** Discussão avançada. Incluem análises formais, comparações entre métodos e pesquisa atual.
- **Seção 11:** Resumo e conexões com outros capítulos.

3.1 O Problema Fundamental: Como Aproveitar Dados Não Rotulados?

3.1.1 A Analogia do Professor e do Estágio

Imagine que você está aprendendo um novo idioma. Você tem: - Um **professor** que pode rotular corretamente algumas frases (dados rotulados) — mas ele é caro e tem pouco tempo. - Milhares de livros, artigos e conversas gravadas (dados não rotulados) — abundantes e gratuitos.

O que você faz? Você estuda as poucas frases rotuladas para aprender as regras básicas e, em seguida, usa os livros não rotulados para praticar, refinar sua compreensão e aprender nuances. Eventualmente, você se torna fluente com muito menos ajuda do professor do que se dependesse apenas dele.

Isso é aprendizado semi-supervisionado: usar a estrutura dos dados não rotulados para amplificar o sinal dos poucos rótulos disponíveis (Bishop; Duda).

3.1.2 O Contraste com Outros Paradigmas

Aspecto	Supervisionado	Não-Supervisionado	Semi-Supervisionado
Dados de treino	Rótulos abundantes	Sem rótulos	Poucos rótulos + muitos não rotulados
Custo de rotulagem	Alto	Zero	Baixo (apenas alguns rótulos)
Objetivo	Predição precisa	Descoberta de estrutura	Predição com poucos rótulos
Exemplo	Classificar 10^6 e-mails rotulados	Agrupar 10^6 e-mails sem rótulos	Classificar com 10^3 rótulos + 10^6 não rotulados

Tabela 3.1: Comparação entre paradigmas de aprendizado. Fonte: Bishop; Sorvisto

3.1.3 A Motivação Econômica e Prática

Em muitas aplicações do mundo real, rotular dados é **extremamente caro e lento** (Sorsvsto; Duda):

- **Diagnóstico médico:** cada imagem de raio-X precisa ser analisada por um radiologista especializado.
- **Detecção de fraude:** identificar transações fraudulentas requer investigação humana.
- **NLP jurídico:** rotular cláusulas contratuais exige advogados experientes.
- **Análise de sentimentos:** rotular milhões de tweets manualmente é inviável.

Por outro lado, dados brutos não rotulados são **abundantes e baratos**: imagens da internet, textos de domínio público, logs de transações, etc.

O SSL surge como uma solução prática: use **poucos rótulos** (caros) + **muitos dados não rotulados** (baratos) para obter desempenho próximo ao de modelos treinados com muitos rótulos (Sorsvsto).

3.1.4 Exemplo Didático: O Problema dos Dígitos

Suponha que você tem 1000 imagens de dígitos manuscritos (0 a 9), mas apenas 10 delas estão rotuladas (uma de cada dígito). As outras 990 não têm rótulo.

O que o SSL pode fazer?

1. Use as 10 imagens rotuladas para treinar um classificador inicial (muito fraco, pois só viu um exemplo por classe).
2. Use o classificador para “rotular” as 990 imagens não rotuladas (pseudo-rótulos).
3. Identifique as regiões de alta densidade nos dados: se várias imagens não rotuladas são muito similares entre si e próximas de uma imagem rotulada, provavelmente pertencem à mesma classe.
4. Refine as fronteiras de decisão usando a estrutura dos dados não rotulados.
5. Re-treine o classificador com os pseudo-rótulos mais confiáveis.
6. Repita iterativamente.

No final, o modelo pode alcançar alta acurácia usando apenas 10 rótulos reais (Duda; Mitchell).

3.2 Intuição Geométrica: A Hipótese do Cluster

3.2.1 A Hipótese Fundamental do SSL

A maioria dos algoritmos de SSL assume a **hipótese do cluster** (*cluster assumption*): pontos que estão no mesmo cluster (região de alta densidade) tendem a pertencer à mesma classe (Duda; Bishop).

Em outras palavras:

- As fronteiras de decisão devem passar por regiões de **baixa densidade** de dados.
- As classes formam “ilhas” separadas por “oceanos” de baixa densidade.

```
Região de alta densidade (Classe A)
  * * *
    * * * *
  *   * * *   *
      * * *
      * * *

--- fronteira de decisão ---
(passa por região de baixa densidade)

Região de alta densidade (Classe B)
  + + +
    + + + +
  +   + + +   +
      + + +
      + + +
```

3.2.2 Por que isso funciona matematicamente?

Se a distribuição marginal $p(x)$ tem modas bem separadas (clusters), e cada moda está associada a uma única classe, então conhecer $p(x)$ ajuda a inferir $p(y|x)$ (Bishop). O SSL usa os dados não rotulados para estimar $p(x)$ e, assim, “guiar” a fronteira de decisão para regiões de baixa densidade.

3.2.3 A Analogia do Mapa Topográfico

Imagine um mapa topográfico com montanhas (alta densidade) e vales (baixa densidade). As classes são como espécies de plantas que vivem apenas em montanhas específicas.

- Você tem poucas observações rotuladas: sabe que a espécie A vive na montanha 1 e a espécie B na montanha 2. - Você tem muitas observações não rotuladas: sabe onde estão as montanhas (alta densidade) e os vales (baixa densidade).

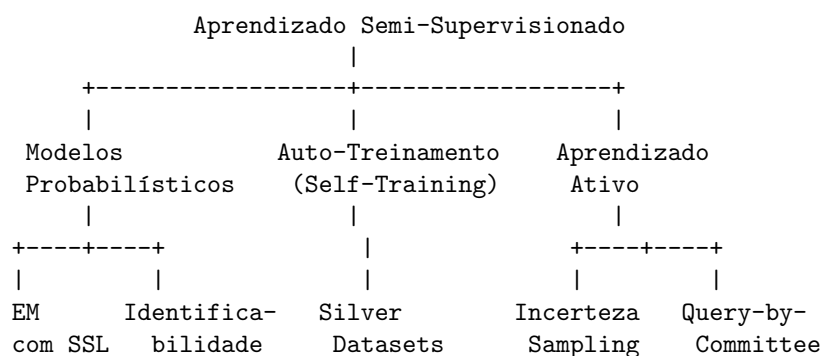
Mesmo sem ver todas as plantas, você pode inferir que toda a montanha 1 provavelmente contém a espécie A , e toda a montanha 2, a espécie B . As fronteiras entre as espécies devem passar pelos vales (baixa densidade).

3.3 Vocabulário Essencial e Mapas Conceituais

3.3.1 Termos que Você Precisa Saber

- **SSL (Semi-Supervised Learning)**: aprendizado que usa dados rotulados e não rotulados.
- **Cluster assumption**: hipótese de que pontos no mesmo cluster pertencem à mesma classe.
- **Manifold assumption**: hipótese de que os dados estão em uma variedade de baixa dimensão.
- **Pseudo-rótulo**: rótulo atribuído automaticamente a um dado não rotulado por um modelo.
- **Self-training (auto-treinamento)**: método iterativo que usa pseudo-rótulos para re-treinar o modelo.
- **Co-training**: método que usa múltiplas “visões” (conjuntos de características) dos dados.
- **Silver dataset**: conjunto de dados rotulado automaticamente (por um modelo) para treinar outro modelo.
- **Aprendizado ativo**: variante onde o algoritmo escolhe quais dados não rotulados enviar para rotulagem humana.
- **Identificabilidade**: em modelos de mistura, capacidade de atribuir corretamente componentes às classes reais.

3.3.2 Mapa Conceitual das Relações



3.3.3 Quando Usar SSL?

Situação	Método SSL recomendado	Razão
Poucos rótulos ($\sim 1\%$ dos dados)	Auto-treinamento + EM	Aproveita estrutura de densidade
Duas ou mais visões dos dados	Co-training	Cada visão ensina a outra
Oráculo humano disponível (caro)	Aprendizado ativo	Minimiza número de rotulagens
Modelo probabilístico (GMM)	EM com rótulos parciais	Identificabilidade garantida
Dados em fluxo (streaming)	Self-training incremental	Adapta-se a novos padrões

Tabela 3.2: Orientação para escolha do método SSL.

3.4 Exemplo Didático Passo-a-Passo

3.4.1 Problema: Classificação com Poucos Rótulos

Vamos usar um exemplo simplificado para entender o auto-treinamento.

Dados (2D, duas classes): - 10 pontos no total: 8 não rotulados, 2 rotulados (um de cada classe) - Classe vermelha (+1): ponto rotulado $R = (1, 1)$ - Classe azul (-1): ponto rotulado $B = (5, 5)$ - 8 pontos não rotulados: $(1.2, 1.1)$, $(0.9, 1.3)$, $(1.1, 0.8)$, $(4.8, 5.2)$, $(5.1, 4.9)$, $(5.3, 5.1)$, $(2.5, 2.5)$, $(2.8, 2.7)$

Passo 1: Treinar um classificador inicial com apenas os 2 pontos rotulados. - Um classificador muito simples: um limite linear pode ser traçado. Como R e B estão distantes, o classificador provavelmente coloca o limite no meio (em $x \approx 3$ ou $y \approx 3$).

Passo 2: Usar o classificador para rotular os 8 pontos não rotulados. - Pontos próximos a R : $(1.2, 1.1)$, $(0.9, 1.3)$, $(1.1, 0.8)$ recebem pseudo-rótulo vermelho (alta confiança). - Pontos próximos a B : $(4.8, 5.2)$, $(5.1, 4.9)$, $(5.3, 5.1)$ recebem pseudo-rótulo azul (alta confiança). - Pontos no meio: $(2.5, 2.5)$, $(2.8, 2.7)$ — o classificador pode ter baixa confiança (próximo à fronteira). Talvez os excluamos da re-treinamento.

Passo 3: Re-treinar o classificador com os 2 rótulos originais + 6 pseudo-rótulos de alta confiança. - Agora o classificador tem 8 pontos para aprender. A fronteira de decisão se torna mais precisa, possivelmente separando melhor os dois clusters.

Passo 4: Repetir: usar o novo classificador para re-rotular os 2 pontos do meio (que antes tinham baixa confiança). - Com um modelo melhor, esses pontos agora podem ser classificados com maior confiança e adicionados ao conjunto de treinamento.

Resultado: Após algumas iterações, o classificador atinge alta acurácia usando apenas 2 rótulos reais (Duda; Mitchell).

3.5 Aprofundamento: Modelos Probabilísticos e EM

3.5.1 O Algoritmo EM para SSL

O algoritmo *Expectation-Maximization* (EM) é fundamental para SSL quando os dados são modelados como uma mistura de distribuições (Duda; Bishop). No contexto SSL, os rótulos ausentes são tratados como **variáveis latentes**.

Seja $p(x, t|\theta) = \sum_{k=1}^K \pi_k p(x, t|\theta_k)$ um modelo de mistura. Para dados rotulados, t é observado; para dados não rotulados, t é latente.

3.5.2 EM com Dados Parcialmente Rotulados

Dados: $\mathcal{L} = \{(x_n, t_n)\}_{n=1}^L$ (rotulados) e $\mathcal{U} = \{x_n\}_{n=L+1}^{L+U}$ (não rotulados).

Passo E (Expectation): Para cada amostra não rotulada x_u , calcule a probabilidade posterior da classe (responsabilidade):

$$r_{uk} = p(t_u = k|x_u, \theta) = \frac{\pi_k p(x_u|\theta_k)}{\sum_{j=1}^K \pi_j p(x_u|\theta_j)}$$

Para amostras rotuladas, $r_{nk} = \mathbf{1}[t_n = k]$ (determinístico).

Passo M (Maximization): Atualize os parâmetros usando as responsabilidades:

$$\begin{aligned}\pi_k &= \frac{\sum_{n=1}^{L+U} r_{nk}}{N} \\ \mu_k &= \frac{\sum_{n=1}^{L+U} r_{nk} x_n}{\sum_{n=1}^{L+U} r_{nk}} \\ \Sigma_k &= \frac{\sum_{n=1}^{L+U} r_{nk} (x_n - \mu_k)(x_n - \mu_k)^T}{\sum_{n=1}^{L+U} r_{nk}}\end{aligned}$$

Onde $N = L + U$ (Bishop; Duda).

3.5.3 Identificabilidade em Misturas

Um dos maiores desafios em modelos de mistura puramente não-supervisionados é a **falta de identificabilidade**: sem rótulos, os componentes da mistura podem ser rotulados de forma arbitrária (permutação). Por exemplo, em uma mistura de duas Gaussianas, trocar os componentes não muda a verossimilhança.

A inclusão de **apenas alguns exemplos rotulados** quebra essa simetria, fixando a correspondência entre componentes e classes reais (Duda). Isso é crucial para o sucesso do SSL em modelos probabilísticos.

3.6 Aprofundamento: Auto-Treinamento (Self-Training)

3.6.1 Algoritmo Genérico

O auto-treinamento é um método heurístico simples, mas eficaz (Duda):

1. Treine um classificador inicial f_0 usando os dados rotulados $\mathcal{L}$.
2. Enquanto não houver convergência:
 - (a) Use f_t para rotular os dados não rotulados $\mathcal{U}$, obtendo pseudo-rótulos.
 - (b) Selecione um subconjunto $\mathcal{U}_{\text{conf}} \subset \mathcal{U}$ dos pontos com maior confiança (ex: > 0.9 de probabilidade).
 - (c) Adicione $\mathcal{U}_{\text{conf}}$ aos dados rotulados: $\mathcal{L} \leftarrow \mathcal{L} \cup \mathcal{U}_{\text{conf}}$.
 - (d) Re-treine f_{t+1} no novo $\mathcal{L}$.
 - (e) Remova $\mathcal{U}_{\text{conf}}$ de $\mathcal{U}$.

3.6.2 Métricas de Confiança

A confiança pode ser medida de várias formas (Mitchell; Bishop):

- **Probabilidade máxima:** $\max_k p(y = k|x)$ (para classificadores probabilísticos).
- **Margem:** diferença entre a maior e a segunda maior probabilidade.
- **Entropia:** $H(y|x) = -\sum_k p(y = k|x) \log p(y = k|x)$ (menor entropia = maior confiança).

3.6.3 Silver Datasets e Augmented SBERT

Em contextos modernos de NLP, o SSL evoluiu para o uso de *silver datasets* (Alammar). O procedimento *Augmented SBERT* funciona assim:

1. Treine um modelo *Cross-encoder* (lento, preciso) em um pequeno conjunto *Gold* (rotulado por humanos).
2. Use esse modelo para rotular massivamente um grande conjunto de dados não rotulados, gerando o conjunto *Silver*.
3. Treine um modelo *Bi-encoder* (rápido, eficiente) no conjunto *Silver*.
4. O modelo final pode ser usado em produção (alta velocidade) com desempenho próximo ao do *Cross-encoder*.

Esse é um exemplo clássico de *knowledge distillation* combinado com SSL (Alammar; Sorvisto).

3.7 Aprofundamento: Aprendizado Ativo (Active Learning)

3.7.1 Definição e Motivação

O aprendizado ativo é um caso especial de SSL onde o algoritmo tem **controle** sobre quais exemplos não rotulados devem ser enviados para um *oráculo* (especialista humano) para rotulagem (Mitchell; Duda). O objetivo é minimizar o número de rotulagens necessárias para atingir um determinado desempenho.

3.7.2 Estratégias de Seleção

As principais estratégias de seleção são (Mitchell; Bishop):

1. **Uncertainty sampling (amostragem por incerteza):** Selecione os pontos nos quais o modelo tem maior incerteza.

$$x^* = \arg \max_x H(y|x) = \arg \max_x \left(- \sum_k p(y = k|x) \log p(y = k|x) \right)$$

2. **Query-by-committee (consulta por comitê):** Use um ensemble de modelos; selecione os pontos onde há maior discordância entre eles.

$$x^* = \arg \max_x \frac{1}{M} \sum_{m=1}^M (y_m(x) - \bar{y}(x))^2$$

3. **Expected model change (mudança esperada do modelo):** Selecione os pontos que mais alterariam o modelo atual.
4. **Expected error reduction (redução esperada do erro):** Selecione os pontos que mais reduziriam o erro esperado em dados não rotulados.

3.7.3 Aprendizado Ativo vs. SSL Passivo

Aspecto	SSL Passivo	Aprendizado Ativo
Papel do algoritmo	Recebe dados (rotulados + não rotulados)	Seleciona quais dados rotular
Interação humana	Nenhuma (usa pseudo-rótulos)	Sim (oráculo rotula exemplos selecionados)
Eficiência de rótulos	Média	Alta (menos rótulos necessários)
Complexidade	Baixa	Média (requer re-treinamento frequente)
Melhor para	Cenários com muitos dados não rotulados	Cenários onde rotulagem é cara

Tabela 3.3: Comparação entre SSL passivo e aprendizado ativo.

3.8 Análise Formal: Condições para o Sucesso do SSL

3.8.1 Quando o SSL Ajuda?

O SSL é benéfico quando (Bishop; Duda):

1. **A hipótese do cluster é válida:** as classes formam clusters separados.
2. **A distribuição marginal $p(x)$ é informativa sobre $p(y|x)$:** a estrutura dos dados não rotulados é relevante para a classificação.
3. **O número de dados rotulados é muito pequeno:** com muitos rótulos, o ganho do SSL é marginal.

3.8.2 Quando o SSL Pode Ser Prejudicial?

O SSL pode piorar o desempenho em certas situações (Duda; Mitchell):

1. **Violação da hipótese do cluster:** se a fronteira de decisão ideal corta uma região de alta densidade.
2. **Rótulos iniciais não representativos:** se os poucos rótulos disponíveis são atípicos.
3. **Muitos outliers:** dados não rotulados espúrios podem “puxar” a fronteira na direção errada.
4. **Propagação de erros:** pseudo-rótulos incorretos podem se amplificar iterativamente.

3.8.3 Identificabilidade: A Base Teórica

Em modelos de mistura, a falta de identificabilidade é o principal obstáculo teórico. Formalmente, um modelo é identificável se:

$$p(x|\theta) = p(x|\theta') \Rightarrow \theta = \theta' \quad (\text{a menos de permutação de componentes})$$

Para misturas de Gaussianas, sem rótulos, a permutação dos componentes produz a mesma verossimilhança. Com alguns exemplos rotulados, a permutação é fixada, tornando o modelo identificável (Duda). Isso justifica teoricamente o uso de SSL em modelos probabilísticos.

3.9 Comparação Entre Métodos SSL

Característica Aprendizado ativo	EM com rótulos	Auto-treinamento	Co-training
Suposição principal	Modelo de mistura	Classificador confiável	Múltiplas visões
Óráculo disponível			
Requer múltiplas visões?	Não	Não	Sim
Não Iterativo?			
Sim	Sim	Sim	Sim
Garantias teóricas	Sim (convergência local)	Poucas	Sim (sob condições)
Sim (limites de erro)			
Escalabilidade	Média	Alta	Média
Baixa (requer óráculo)			
Custo computacional	Médio	Baixo	Alto (múltiplos modelos)
Médio-alto			

Tabela 3.4: Comparação entre métodos de SSL. Fonte: Bishop; Duda; Mitchell

3.10 Limitações e Quando o SSL Falha

3.10.1 Custos e Desvantagens

1. **Risco de overfitting aos pseudo-rótulos:** se o classificador inicial é muito ruim, os pseudo-rótulos podem ser sistematicamente errados.
2. **Propagação de erros:** erros iniciais se amplificam iterativamente (efeito *snowball*).

3. **Dependência da hipótese do cluster:** se as classes não formam clusters separados, o SSL não ajuda.
4. **Complexidade computacional:** muitos métodos SSL exigem re-treinamento iterativo.

3.10.2 Quando o SSL NÃO Ajuda

- **Fronteira de decisão cruza regiões de alta densidade:** por exemplo, duas classes que se interpenetram.
- **Pouca estrutura nos dados:** se $p(x)$ é aproximadamente uniforme, dados não rotulados não adicionam informação.
- **Rótulos iniciais são enviesados:** se os exemplos rotulados não representam a distribuição real.
- **Alta dimensionalidade:** a hipótese do cluster se torna menos plausível em espaços de alta dimensão.

3.11 Pesquisa Atual e Fronteiras

3.11.1 Dilema Estabilidade-Plasticidade

Em ambientes não estacionários (dados mudam ao longo do tempo), o SSL enfrenta o **dilema estabilidade-plasticidade** (Duda): como adicionar novos padrões a partir de novos dados não rotulados sem esquecer os padrões antigos? Pesquisas atuais exploram algoritmos de agrupamento online e *lifelong learning*.

3.11.2 Redução de Alucinações via RAG

A integração de métodos de geração com recuperação de documentos (*Retrieval-Augmented Generation* — RAG) busca usar SSL para rotular veracidade em tempo real, mitigando previsões incorretas de modelos generativos (Alammar). O SSL ajuda a identificar quando o modelo está “alucinando” (gerando conteúdo falso) ao comparar com dados não rotulados de alta confiança.

3.11.3 Hibridização Indutiva-Analítica

Pesquisas atuais exploram como combinar aprendizado estatístico baseado em dados (indutivo) com conhecimento prévio explícito baseado em regras (analítico) para reduzir a complexidade amostral necessária para generalização (Mitchell). Em SSL, isso significa usar regras de domínio para guiar a atribuição de pseudo-rótulos, reduzindo o risco de propagação de erros.

3.11.4 SSL em LLMs e Auto-Supervisão

Embora o treinamento de LLMs como GPT e Llama seja formalmente *self-supervised learning* (auto-supervisionado) e não semi-supervisionado, os dois paradigmas estão convergindo. O auto-supervisionado gera rótulos automaticamente a partir da estrutura dos dados (ex: próxima palavra), alcançando escala sem precedentes (Raschka). Pesquisas atuais buscam combinar auto-supervisão (pré-treinamento) com SSL (ajuste fino com poucos rótulos) para máxima eficiência.

3.11.5 Deep Semi-Supervised Learning

O uso de redes neurais profundas em SSL é uma área ativa. Métodos como **Pseudo-Labeling**, **Mix-Match**, e **FixMatch** combinam aumento de dados, consistência regularização e pseudo-rótulos para alcançar desempenho próximo ao supervisionado com 1% dos rótulos (Goodfellow).

3.12 Resumo e Conexões com Outros Capítulos

3.12.1 O que Você Deve Lembrar

1. **Ideia central:** usar dados não rotulados para amplificar o sinal de poucos rótulos.
2. **Hipótese do cluster:** fronteiras de decisão devem passar por regiões de baixa densidade.
3. **EM com rótulos:** abordagem probabilística rigorosa, resolve identificabilidade.
4. **Auto-treinamento:** heurística simples e eficaz (pseudo-rótulos iterativos).

5. **Aprendizado ativo:** algoritmo escolhe quais pontos rotular (minimiza custo).
6. **Silver datasets:** rotulagem massiva automática para treinar modelos eficientes.

3.12.2 Conexões com Outros Capítulos

Este capítulo sobre Aprendizado Semi-Supervisionado se conecta com os demais capítulos do resumo das seguintes formas:

- **Ensemble Learning (Capítulo 1):** O *co-training* (mencionado brevemente na Seção 3) utiliza múltiplos classificadores (ensemble) treinados em diferentes visões dos dados. Cada classificador rotula dados não rotulados para o outro, criando um ciclo de aprendizado semi-supervisionado baseado em ensemble.
- **Aprendizado Não-Supervisionado (Capítulo 2):** O SSL é construído diretamente sobre clustering e estimativa de densidade. A hipótese do cluster (*cluster assumption*) — de que pontos no mesmo cluster tendem a ter a mesma classe — é a base teórica que justifica o uso de dados não rotulados. Algoritmos de clustering (K-means, hierárquico) e estimativa de densidade (Parzen, KNN) são frequentemente usados como sub-rotinas em métodos SSL.
- **Aprendizado por Transferência (Capítulo 4):** O auto-supervisionado (*self-supervised learning*) — que compartilha a letra “S” com SSL — é usado no pré-treinamento de modelos de fundação (ex: BERT, GPT). A combinação de pré-treinamento auto-supervisionado (Capítulo 4) com ajuste fino semi-supervisionado (poucos rótulos) é uma prática comum e poderosa em NLP.
- **Otimização de Modelos (Capítulo 5):** O algoritmo *Expectation-Maximization* (EM) com rótulos parciais — apresentado na Seção 5 — é um método de otimização para variáveis latentes. O Capítulo 5 aborda fundamentos de otimização (gradiente, Newton, etc.) que são essenciais para entender o funcionamento do EM.

Resumo das relações:

Capítulo	Conexão com Semi-Supervisionado
Ensemble (1)	Co-training (ensemble de classificadores)
Não-Supervisionado (2)	Clustering, hipótese do cluster, EM
Transferência (4)	Auto-supervisionado como pré-treinamento
Otimização (5)	EM como algoritmo de otimização

3.12.3 Cheat Sheet para Revisão Rápida (Leitores Experientes)

Quero...	Uso...	Fórmula chave
Modelo probabilístico rigoroso	EM com rótulos parciais	$r_{uk} = \frac{\pi_k p(x_u \theta_k)}{\sum_j \pi_j p(x_u \theta_j)}$
Heurística simples e rápida	Auto-treinamento	Pseudo-rótulos de alta confiança
Minimizar rotulagens humanas	Aprendizado ativo	$x^* = \arg \max_x H(y x)$
Duas visões dos dados	Co-training	Concordância entre visões
Rotulagem massiva automática	Silver datasets	Cross-encoder $\rightarrow$ Bi-encoder

Tabela 3.5: Resumo rápido para consulta.

Capítulo 4

Aprendizado por Transferência

Visão Geral do Capítulo

Este capítulo apresenta o Aprendizado por Transferência (*Transfer Learning*), um paradigma que revolucionou o campo da inteligência artificial nos últimos anos. A ideia central é reutilizar conhecimento adquirido em uma tarefa (fonte) para auxiliar o aprendizado em outra tarefa (alvo), geralmente com muito menos dados e computação.

O capítulo está organizado para atender tanto leitores que buscam uma primeira exposição ao tema quanto aqueles que desejam uma revisão técnica aprofundada.

- **Seções 1 a 4:** Para todos os leitores. Apresentam intuição, motivação, analogias e exemplos didáticos.
- **Seções 5 a 7:** Aprofundamento técnico. Contêm equações, algoritmos e derivações.
- **Seções 8 a 10:** Discussão avançada. Incluem análises formais, comparações entre métodos e pesquisa atual.
- **Seção 11:** Resumo e conexões com outros capítulos.

4.1 O Problema Fundamental: Como Aprender com Poucos Dados?

4.1.1 A Analogia do Poliglota

Imagine que você já fala português fluentemente e decide aprender espanhol. Você não começa do zero: já conhece a estrutura de frases, conceitos de gênero e número, e muitas palavras são semelhantes (cognatos). Você **transfere** o conhecimento do português para o espanhol.

Em vez de aprender 10.000 palavras do zero, você precisa aprender apenas as diferenças (cerca de 2.000 palavras e algumas regras gramaticais). O tempo de aprendizado cai drasticamente.

Isso é aprendizado por transferência: usar conhecimento prévio para acelerar o aprendizado em uma nova tarefa relacionada (Bishop; Raschka).

4.1.2 O Contraste com o Paradigma Tradicional

Aspecto	Aprendizado Tradicional	Aprendizado por Transferência
Ponto de partida	Pesos aleatórios	Pesos de modelo pré-treinado
Dados necessários	Grandes quantidades ($10^5 - 10^9$)	Pequenas quantidades ($10^2 - 10^4$)
Custo computacional	Alto (dias ou semanas)	Baixo (horas ou minutos)
Generalização	Depende da quantidade de dados	Boa mesmo com poucos dados

Tabela 4.1: Comparação entre aprendizado tradicional e por transferência.

4.1.3 A Motivação Econômica e Prática

Na prática, o aprendizado por transferência é **essencial** para viabilizar projetos de IA (Sorvisto; Raschka):

- **Custo de treinamento:** Treinar o GPT-3 do zero custou cerca de **US\$ 4,6 milhões** em computação. O ajuste fino (*fine-tuning*) custa centenas de dólares.
- **Dados rotulados escassos:** Em domínios como medicina ou direito, obter 10.000 exemplos rotulados pode ser inviável.
- **Tempo de desenvolvimento:** Em vez de meses de treinamento, o ajuste fino pode ser feito em horas.

Sem transferência, apenas grandes empresas de tecnologia teriam recursos para desenvolver modelos de ponta. Com transferência, qualquer desenvolvedor pode adaptar modelos massivos para suas tarefas específicas (Raschka; Alammari).

4.1.4 Exemplo Didático: Reconhecimento de Cachorros

Suponha que você quer classificar imagens em “cachorro” ou “não cachorro”, mas tem apenas 100 imagens rotuladas.

Sem transferência: - Treine uma rede neural do zero com 100 imagens. - A rede provavelmente terá overfitting (decorará as 100 imagens, não generalizará).

Com transferência: - Use um modelo pré-treinado no ImageNet (14 milhões de imagens, 1000 classes). - Este modelo já sabe reconhecer bordas, texturas, formas de olhos, orelhas, patas. - Substitua a última camada (que previa 1000 classes) por uma nova camada que prevê 2 classes (“cachorro” ou “não cachorro”). - Re-treine apenas as últimas camadas com suas 100 imagens. - Resultado: alta acurácia, mesmo com poucos dados.

4.2 Intuição Geométrica: A Hierarquia de Características

4.2.1 A Analogia da Pirâmide de Conhecimento

Redes neurais profundas aprendem características em uma hierarquia (Bishop; Goodfellow):

Camadas Finais (específicas da tarefa)

Classes específicas (cachorro,
gato, carro, etc.)

Camadas Intermediárias (semânticas)

Partes de objetos (olhos, rodas,
patas, janelas)

Camadas Iniciais (genéricas)

Bordas, cantos, texturas, cores

Imagem de entrada (pixels)

- **Camadas iniciais:** genéricas (bordas, texturas) — úteis para **qualquer** tarefa de visão. - **Camadas intermediárias:** semânticas (olhos, rodas) — úteis para tarefas relacionadas. - **Camadas finais:** específicas (cachorro vs. gato) — devem ser re-treinadas.

Quanto mais genérica a característica, mais transferível ela é entre tarefas (Bishop; Goodfellow).

4.2.2 Por que Transferir Funciona?

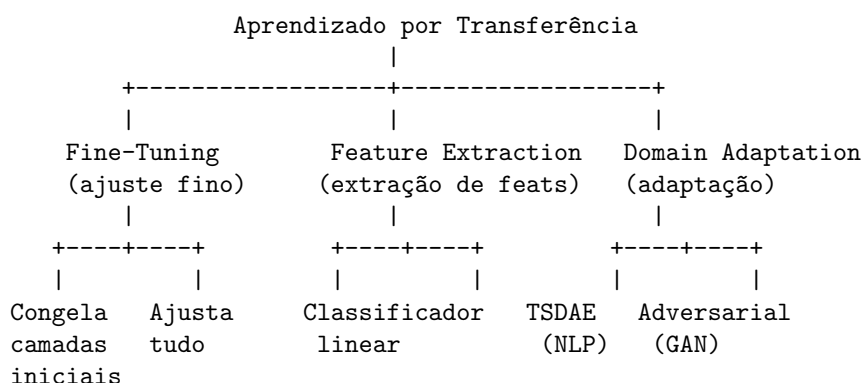
Formalmente, se duas tarefas compartilham a mesma distribuição marginal $p(x)$ (ou pelo menos as mesmas características de baixo nível), então as camadas iniciais de uma rede treinada em uma tarefa serão úteis para a outra (Bishop). Isso é particularmente verdadeiro em visão computacional (todas as imagens têm bordas, cores, texturas) e em NLP (todas as línguas têm sintaxe e semântica).

4.3 Vocabulário Essencial e Mapas Conceituais

4.3.1 Termos que Você Precisa Saber

- **Transfer Learning:** reutilização de conhecimento de uma tarefa fonte para uma tarefa alvo.
- **Pré-treinamento (pretraining):** treinar um modelo em uma tarefa genérica com dados abundantes.
- **Ajuste fino (fine-tuning):** continuar o treinamento do modelo pré-treinado na tarefa alvo.
- **Modelo de fundação (foundation model):** modelo pré-treinado em larga escala (ex: GPT, BERT, ViT).
- **Congelamento de camadas (layer freezing):** fixar os pesos das camadas iniciais para não serem atualizados.
- **Cabeça (head):** camadas finais específicas da tarefa (substituídas no fine-tuning).
- **PEFT (Parameter-Efficient Fine-Tuning):** métodos que ajustam apenas uma fração pequena dos parâmetros.
- **LoRA (Low-Rank Adaptation):** técnica PEFT que injeta matrizes de baixo posto no modelo.
- **Adaptação de domínio (domain adaptation):** transferência onde a distribuição dos dados muda.
- **Destilação de conhecimento (knowledge distillation):** transferência de um modelo grande (professor) para um pequeno (aluno).

4.3.2 Mapa Conceitual das Relações



4.3.3 Quando Usar Transferência?

Situação	Abordagem	Razão
Poucos dados na tarefa alvo ($< 10^3$)	Extração de características (congelar tudo)	Risco de overfitting no fine-tuning
Dados moderados ($10^3 - 10^5$)	Fine-tuning de camadas finais	Permite adaptação sem overfitting
Muitos dados ($> 10^5$)	Fine-tuning completo	Dados suficientes para ajustar tudo
Recursos computacionais limitados	PEFT (LoRA)	Ajusta apenas $< 1\%$ dos parâmetros
Domínio alvo muito diferente	Adaptação de domínio + fine-tuning	Distribuições significativamente diferentes

Tabela 4.2: Orientação para escolha da abordagem de transferência.

4.4 Exemplo Didático Passo-a-Passo: Fine-Tuning de um Classificador de Imagens

4.4.1 Problema: Classificar Raças de Cachorros

Suponha que você quer classificar imagens em 3 raças de cachorros: Labrador, Poodle e Buldogue. Você tem apenas 300 imagens (100 por raça). Um modelo treinado do zero com 300 imagens teria overfitting.

Solução: usar um modelo pré-treinado no ImageNet (resnet50, por exemplo).

4.4.2 Passo a Passo

Passo 1: Carregar o modelo pré-treinado.

O modelo resnet50 tem a seguinte arquitetura (simplificada):

Entrada (224x224x3)

↓

Camadas convolucionais iniciais (bordas, texturas)

↓

... (muitas camadas)

↓

Camada final (fully connected) com 1000 saídas (classes do ImageNet)

Passo 2: Substituir a cabeça de classificação.

Remova a última camada (saída de 1000 classes) e substitua por uma nova camada com 3 saídas (Labrador, Poodle, Buldogue).

Passo 3: Congelar camadas iniciais (opcional).

Para evitar overfitting com 300 imagens, congele as primeiras camadas (que detectam bordas e texturas). Apenas as últimas camadas serão re-treinadas.

Passo 4: Fine-tuning.

Treine o modelo com suas 300 imagens, mas com uma taxa de aprendizado menor (ex: 10^{-5} em vez de 10^{-3}) para não destruir o conhecimento prévio.

Passo 5: Avaliação.

O modelo atinge acurácia de 85% nas raças — um resultado excelente para apenas 300 imagens. Um modelo treinado do zero com os mesmos 300 dados teria acurácia de cerca de 50% (chute aleatório) (Raschka; Sorvisto).

4.5 Aprofundamento: Fine-Tuning e Congelamento de Camadas

4.5.1 Fine-Tuning: Formulação Matemática

Seja $f_\theta(x)$ um modelo pré-treinado com parâmetros θ_0 . No fine-tuning, resolvemos:

$$\theta^* = \arg \min_{\theta} \frac{1}{N_{\text{target}}} \sum_{n=1}^{N_{\text{target}}} \mathcal{L}(f_\theta(x_n), t_n) + \lambda \|\theta - \theta_0\|^2$$

O termo de regularização $\lambda \|\theta - \theta_0\|^2$ (opcional) penaliza desvios muito grandes dos pesos pré-treinados, ajudando a evitar overfitting quando os dados alvo são escassos (Bishop; Goodfellow).

4.5.2 Congelamento de Camadas

Seja $\theta = \{\theta_{\text{early}}, \theta_{\text{late}}\}$, onde θ_{early} são camadas iniciais (genéricas) e θ_{late} são camadas finais (específicas). No congelamento:

$$\begin{aligned} \theta_{\text{early}}^* &= \theta_{\text{early}}^{(0)} \quad (\text{fixos}) \\ \theta_{\text{late}}^* &= \arg \min_{\theta_{\text{late}}} \frac{1}{N} \sum_{n=1}^N \mathcal{L}(f_{\theta_{\text{early}}^{(0)}, \theta_{\text{late}}} (x_n), t_n) \end{aligned}$$

Apenas θ_{late} é atualizado. O número de épocas para fine-tuning geralmente é pequeno (5 a 20) comparado ao pré-treinamento (centenas de épocas) (Raschka).

4.5.3 Taxa de Aprendizado Diferenciada

Em vez de congelar completamente, podemos usar taxas de aprendizado diferentes (Bishop; Goodfellow):

$$\begin{aligned} \eta_{\text{early}} &= \eta_0 \cdot \gamma \quad (\text{menor}) \\ \eta_{\text{late}} &= \eta_0 \quad (\text{maior}) \end{aligned}$$

Onde $\gamma \in [0.01, 0.1]$ é um fator de redução. Isso permite que as camadas iniciais se adaptem lentamente (apenas se os dados alvo forem suficientes), enquanto as camadas finais se adaptam rapidamente.

4.6 Aprofundamento: PEFT e LoRA

4.6.1 O Problema do Fine-Tuning Completo

Para modelos massivos como GPT-3 (175 bilhões de parâmetros), o fine-tuning completo é inviável: - Armazenar uma cópia dos pesos para cada tarefa requer terabytes de memória. - O custo computacional do fine-tuning completo é quase tão alto quanto o pré-treinamento.

4.6.2 LoRA (Low-Rank Adaptation)

O LoRA (Hu et al., 2021) injeta matrizes de baixo posto nas camadas do modelo (Raschka; Alammari). Para uma camada com pesos $W \in \mathbb{R}^{d \times d}$, o LoRA adiciona:

$$W' = W + \Delta W = W + BA$$

Onde: - $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times d}$, com $r \ll d$ (tipicamente $r = 4, 8$, ou 16) - A é inicializada aleatoriamente (distribuição Gaussiana) - B é inicializada como zero ($\Delta W = 0$ no início)

Durante o fine-tuning, apenas A e B são atualizados — W permanece congelado. O número de parâmetros treináveis cai de d^2 para $2dr$, uma redução de $\frac{2r}{d}$. Para $d = 4096$ e $r = 8$, a redução é de $\frac{16}{4096} = 0.39\%$ (apenas 0.39% dos parâmetros são ajustados!).

4.6.3 Vantagens do LoRA

1. **Eficiência de memória:** para K tarefas, armazene apenas $B_k A_k$ (pequeno) para cada tarefa, compartilhando o modelo base W .
2. **Nenhuma latência adicional na inferência:** após o treinamento, podemos computar $W' = W + BA$ e armazenar o modelo fundido.
3. **Qualidade comparável:** LoRA atinge desempenho similar ao fine-tuning completo em muitas tarefas (Raschka; Alammr).

4.7 Aprofundamento: Destilação de Conhecimento (Knowledge Distillation)

4.7.1 Definição Formal

A destilação de conhecimento transfere o conhecimento de um modelo **professor** (grande, lento, preciso) para um modelo **aluno** (pequeno, rápido, eficiente) (Hinton et al., 2015; Sorvisto).

Seja: - $p_{\text{teacher}}(y|x)$: distribuição de saída do professor - $p_{\text{student}}(y|x)$: distribuição de saída do aluno
A função de perda do aluno é uma combinação (Bishop; Goodfellow):

$$\mathcal{L}_{\text{student}} = \alpha \cdot \underbrace{\text{KL}(p_{\text{teacher}} \| p_{\text{student}})}_{\text{Perda de destilação}} + (1 - \alpha) \cdot \underbrace{\text{CE}(y_{\text{true}}, p_{\text{student}})}_{\text{Perda supervisionada padrão}}$$

Onde: - KL é a divergência de Kullback-Leibler - CE é a entropia cruzada com os rótulos verdadeiros
- $\alpha \in [0, 1]$ controla o trade-off

4.7.2 Temperatura na Destilação

Para suavizar as probabilidades do professor, usa-se uma **temperatura** $T > 1$:

$$p_{\text{teacher}}^{(T)}(y|x) = \frac{\exp(z_y/T)}{\sum_j \exp(z_j/T)}$$

Temperaturas mais altas ($T = 4$ ou $T = 10$) produzem distribuições mais suaves, revelando relações entre classes (ex: o professor pode dizer “isso é 90% um labrador, 8% um golden retriever, 2% um buldogue”).

4.8 Análise Formal: Condições para Transferência Bem-Sucedida

4.8.1 Similaridade entre Tarefas

A transferência é bem-sucedida quando as tarefas fonte e alvo compartilham características relevantes. Formalmente (Bishop; Goodfellow), se:

$$d(p_{\text{fonte}}(x, y), p_{\text{alvo}}(x, y)) \text{ é pequena}$$

para alguma métrica de distância entre distribuições (ex: divergência KL, distância de Wasserstein), então o conhecimento transferível é alto.

4.8.2 Teoria de Generalização

O erro de generalização no fine-tuning pode ser limitado por (Bishop; Goodfellow):

$$\epsilon_{\text{alvo}} \leq \epsilon_{\text{fonte}} + d(p_{\text{fonte}}, p_{\text{alvo}}) + \sqrt{\frac{C}{N_{\text{alvo}}}}$$

Onde: - ϵ_{fonte} : erro na tarefa fonte - $d(p_{\text{fonte}}, p_{\text{alvo}})$: distância entre as distribuições - C/N_{alvo} : termo de complexidade amostral

Isso formaliza a intuição: quanto mais similares as tarefas, e quanto mais dados alvo, melhor a generalização.

4.9 Comparação Entre Métodos de Transferência

Método	Parâmetros treináveis	Custo	Overfitting	Quando usar
Extração de características	Cabeça apenas	Muito baixo	Baixo	Muito poucos dados
Fine-tuning (camadas finais)	Camadas superiores	Baixo	Médio	Dados moderados
Fine-tuning completo	Todos	Alto	Alto	Muitos dados
LoRA	$2dr$ (muito poucos)	Baixo	Baixo	LLMs, múltiplas tarefas
Destilação	Modelo aluno (pequeno)	Médio	Baixo	Implantação em produção
Adaptação de domínio	Especializado	Médio-Alto	Médio	Dados fonte e alvo diferentes

Tabela 4.3: Comparação entre métodos de aprendizado por transferência. Fonte: Raschka; Alammari; Sorvisto

4.10 Limitações e Quando a Transferência Falha

4.10.1 Custos e Desvantagens

1. **Transferência negativa:** se as tarefas fonte e alvo são muito diferentes, o conhecimento prévio pode **prejudicar** o desempenho.
2. **Overfitting aos dados fonte:** o modelo pode ter aprendido vieses do conjunto fonte que não se aplicam ao alvo.
3. **Custo do pré-treinamento:** treinar modelos de fundação ainda é extremamente caro (apenas grandes empresas podem fazê-lo).
4. **Questões de licenciamento:** muitos modelos pré-treinados têm licenças restritivas para uso comercial.

4.10.2 Quando a Transferência NÃO Ajuda

- **Tarefas radicalmente diferentes:** transferir de reconhecimento de imagens médicas para classificação de texto.
- **Domínios com características diferentes:** imagens RGB vs. imagens de satélite multiespectrais.
- **Dados fonte de baixa qualidade:** se o modelo pré-treinado tem vieses ou erros sistemáticos.
- **Pouca similaridade semântica:** transferir de classificação de animais para classificação de veículos pode funcionar (bordas, texturas), mas de animais para diagnóstico médico provavelmente não.

4.11 Pesquisa Atual e Fronteiras

4.11.1 Modelos de Fundação e Escalabilidade

Os **modelos de fundação** (foundation models) são modelos massivos pré-treinados em dados diversificados (ex: GPT-4, Llama, Gemini) que servem como base para inúmeras tarefas (Raschka; Alammr). A pesquisa atual foca em:

- **Mistura de especialistas (MoE):** ativar apenas parte dos parâmetros por token para escalar sem aumentar custo.
- **Contextos longos:** modelos que podem processar milhões de tokens de contexto (ex: Gemini 1.5, Claude 3).
- **Multimodalidade:** modelos que aceitam texto, imagem, áudio e vídeo como entrada.

4.11.2 Aprendizado ao Longo da Vida (Lifelong Learning)

Pesquisas focam em como um agente pode aprender continuamente sobre múltiplas tarefas sem sofrer de **esquecimento catastrófico** — o fenômeno em que o aprendizado de uma nova tarefa degrada o desempenho em tarefas anteriores (Mitchell; Goodfellow). Técnicas como *elastic weight consolidation* (EWC) e *memory replay* são ativamente pesquisadas.

4.11.3 Adaptação de Domínio Não-Supervisionada

Desenvolvimento de técnicas para adaptar modelos a novos domínios (ex: de textos da internet para textos jurídicos) usando apenas dados não rotulados do novo domínio (Alammr). O método **TSDAE** (*Transformer-based Sequential Denoising Auto-Encoder*) é um exemplo recente que atinge estado-da-arte em adaptação de embeddings.

4.11.4 Destilação de Conhecimento para LLMs

Dado o custo de inferência de LLMs, há um foco crescente em destilar modelos gigantes (professores) em modelos pequenos (alunos) que podem rodar em dispositivos de borda (Sorvisto). Métodos como *distilBERT*, *TinyBERT*, e *GPTQ* permitem compressão de até 90% com perda mínima de acurácia.

4.11.5 Hibridização Indutiva-Analítica

Exploração de como combinar conhecimento prévio explícito (regras de domínio) com aprendizado estatístico para guiar a transferência de forma mais precisa em cenários de dados extremamente escassos (Mitchell). Isso é particularmente relevante em domínios regulados (medicina, finanças) onde regras de especialistas estão disponíveis.

4.12 Resumo e Conexões com Outros Capítulos

4.12.1 O que Você Deve Lembrar

1. **Ideia central:** reutilizar conhecimento de uma tarefa fonte para acelerar o aprendizado na tarefa alvo.
2. **Pré-treinamento + Fine-tuning:** paradigma dominante em NLP e visão computacional.
3. **Congelamento de camadas:** preserva características genéricas das camadas iniciais.

4. **PEFT/LoRA**: ajusta $< 1\%$ dos parâmetros, viabilizando LLMs.
5. **Destilação**: transfere conhecimento de modelos grandes para pequenos (implantação).
6. **Adaptação de domínio**: transferência onde a distribuição dos dados muda.

4.12.2 Conexões com Outros Capítulos

Este capítulo sobre Aprendizado por Transferência se conecta com os demais capítulos do resumo das seguintes formas:

- **Ensemble Learning (Capítulo 1)**: A destilação de conhecimento (*knowledge distillation*) — apresentada na Seção 7 — transfere o conhecimento de um ensemble de professores (ou um modelo grande) para um modelo aluno menor. Esta é uma forma de transferência em que o “professor” pode ser um ensemble, conectando diretamente os dois capítulos.
- **Aprendizado Não-Supervisionado (Capítulo 2)**: O pré-treinamento de modelos de fundação (ex: BERT, GPT) utiliza auto-supervisão, que é uma forma de aprendizado não-supervisionado (abordado na Seção 10 do Capítulo 2). O conhecimento adquirido nessa fase é exatamente o que é transferido via fine-tuning neste capítulo.
- **Aprendizado Semi-Supervisionado (Capítulo 3)**: Uma prática comum em NLP é combinar: (1) pré-treinamento auto-supervisionado (Capítulo 4) com (2) fine-tuning semi-supervisionado usando poucos rótulos (Capítulo 3). O auto-supervisionado gera representações genéricas; o SSL as especializa com pouca supervisão.
- **Otimização de Modelos (Capítulo 5)**: O fine-tuning é essencialmente um problema de otimização onde o ponto inicial não é aleatório, mas sim os pesos do modelo pré-treinado. A regularização $\|\mathbf{w} - \mathbf{w}_0\|^2$ (mencionada na Seção 5 deste capítulo) é uma forma de regularização que conecta otimização e transferência. O Capítulo 5 aborda métodos de otimização (AdamW, decaimento de taxa de aprendizado) que são usados no fine-tuning.

Resumo das relações:

Capítulo	Conexão com Transferência
Ensemble (1)	Destilação de conhecimento (ensemble $\rightarrow$ aluno)
Não-Supervisionado (2)	Auto-supervisionado como pré-treinamento
Semi-Supervisionado (3)	Pré-treino auto-supervisionado + fine-tuning SSL
Otimização (5)	Fine-tuning como otimização com inicialização especial

4.12.3 Cheat Sheet para Revisão Rápida (Leitores Experientes)

Quero...	Uso...	Fórmula chave
Reutilizar modelo pré-treinado	Fine-tuning	$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum \mathcal{L}(f_{\theta}(x_n), t_n)$
Evitar overfitting no fine-tuning	Congelamento de camadas	$\theta_{\text{early}}^* = \theta_{\text{early}}^{(0)}$
Ajustar LLM com poucos parâmetros	LoRA	$W' = W + BA, r \ll d$
Transferir conhecimento para modelo pequeno	Destilação	$\mathcal{L} = \alpha \cdot \text{KL}(p_{\text{teacher}} \ p_{\text{student}}) + (1 - \alpha) \cdot \text{CE}$
Adaptar a domínio diferente	Adaptação de domínio	TSDAE, adversarial training

Tabela 4.4: Resumo rápido para consulta.

Capítulo 5

Otimização de Modelos de Machine Learning

Visão Geral do Capítulo

Este capítulo apresenta os fundamentos e técnicas avançadas de otimização em Machine Learning. A otimização é o motor que permite aos modelos aprenderem a partir dos dados: é o processo de ajustar os parâmetros do modelo para minimizar uma função de perda.

O capítulo está organizado para atender tanto leitores que buscam uma primeira exposição ao tema quanto aqueles que desejam uma revisão técnica aprofundada.

- **Seções 1 a 4:** Para todos os leitores. Apresentam intuição, motivação, analogias e exemplos didáticos.
- **Seções 5 a 7:** Aprofundamento técnico. Contêm equações, algoritmos e derivações.
- **Seções 8 a 10:** Discussão avançada. Incluem análises formais, comparações entre métodos e pesquisa atual.
- **Seção 11:** Resumo e conexões com outros capítulos.

5.1 O Problema Fundamental: Como Encontrar o Mínimo?

5.1.1 A Analogia do Andarilho Cego na Montanha

Imagine que você está em uma montanha à noite, completamente no escuro, e quer encontrar o **ponto mais baixo** do vale. Você não pode ver o panorama completo — apenas sentir o declive sob seus pés.

O que você faz? - Você dá um passo na direção em que o terreno **desce mais acentuadamente** (direção do gradiente negativo). - Você repete até não sentir mais descida (mínimo local).

Isso é **descida do gradiente** (*gradient descent*) — o algoritmo de otimização mais fundamental em Machine Learning (Bishop; Duda).

5.1.2 O Paralelo com Aprendizado

Em Machine Learning, os parâmetros do modelo (pesos $\mathbf{w}$) definem uma superfície de erro (função de perda). O objetivo é encontrar o conjunto de parâmetros que minimiza o erro:

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \mathcal{L}(\mathbf{w})$$

A descida do gradiente atualiza os parâmetros na direção oposta ao gradiente:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla \mathcal{L}(\mathbf{w}^{(t)})$$

Onde η é a **taxa de aprendizado** (*learning rate*) — o tamanho do passo (Bishop).

5.1.3 O Contraste Entre Otimização e Generalização

É crucial distinguir (Bishop; Goodfellow):

Aspecto	Otimização	Generalização
Objetivo	Minimizar perda no treino	Minimizar perda em novos dados
Métrica	Perda no conjunto de treino	Perda no conjunto de teste
Problema	Encontrar mínimo (pode ser local)	Evitar overfitting
Ferramentas	Gradiente, momentum, taxa de aprendizado	Regularização, validação cruzada

Tabela 5.1: Otimização vs. Generalização.

5.1.4 Exemplo Didático: Descida do Gradiente em 1D

Considere a função quadrática (convexa) simples:

$$\mathcal{L}(w) = w^2 - 4w + 4 = (w - 2)^2$$

O mínimo global está em $w^* = 2$.

Passo a passo com $\eta = 0.1$, partindo de $w^{(0)} = 0$:

$$\nabla \mathcal{L}(w) = 2w - 4$$

t=0: w=0.00, grad=-4.00, atualização: w = 0 - 0.1*(-4) = 0.40

t=1: w=0.40, grad=2*0.4-4=-3.20, atualização: w = 0.40 - 0.1*(-3.20) = 0.72

t=2: w=0.72, grad=-2.56, w = 0.72 - 0.1*(-2.56) = 0.976

t=3: w=0.976, grad=-2.048, w = 0.976 - 0.1*(-2.048) = 1.1808

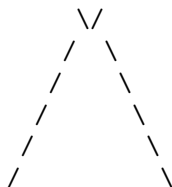
... (aproximando de 2)

Após muitas iterações, w converge para 2 (Bishop; Duda).

5.2 Intuição Geométrica: A Paisagem de Erro

5.2.1 Superfícies Convexas vs. Não-Convexas

Superfície Convexa
(regressão linear)



Um único mínimo global

Superfície Não-Convexa
(redes neurais)



Múltiplos mínimos locais

- **Convexa (ex: regressão linear):** qualquer mínimo local é o mínimo global. A descida do gradiente converge para a solução ótima (Bishop).
- **Não-convexa (ex: redes neurais):** a superfície tem múltiplos vales (mínimos locais). A descida do gradiente pode parar em um vale que não é o mais profundo. Surpreendentemente, em altas dimensões, mínimos locais ruins são raros (Goodfellow).

5.2.2 O Problema dos Platôs e Vales Estreitos

Vale Estreito (alta curvatura)

Platô (curvatura zero)

Gradiente oscila
(precisa de momentum)

Gradiente muito pequeno
(precisa de warmup)

5.2.3 A Analogia da Bola em uma Tábua de Madeira

Imagine uma bola rolando em uma tábua de madeira com diferentes texturas: - **Superfície lisa (função convexa):** a bola rola suavemente até o ponto mais baixo. - **Superfície com ranhuras (vale estreito):** a bola oscila de um lado para o outro antes de descer. - **Superfície plana (platô):** a bola quase não se move. - **Momentum:** como dar um impulso inicial na bola — ela atravessa pequenos obstáculos e mantém a direção.

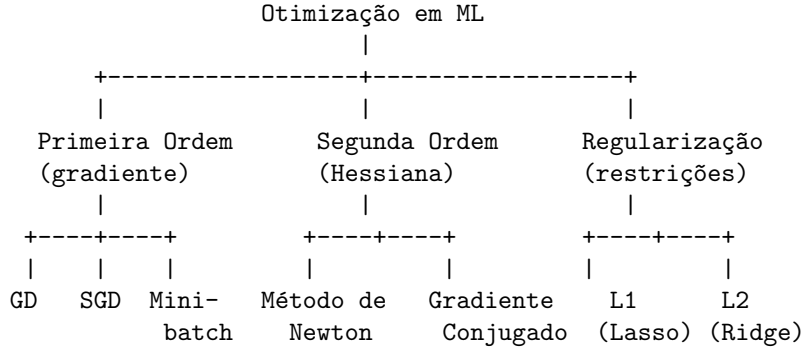
5.3 Vocabulário Essencial e Mapas Conceituais

5.3.1 Termos que Você Precisa Saber

- **Gradiente:** vetor de derivadas parciais da função de perda em relação aos parâmetros. Aponta a direção de maior crescimento.
- **Taxa de aprendizado (learning rate) η :** tamanho do passo na direção do gradiente negativo.
- **Descida do gradiente (gradient descent):** algoritmo iterativo que atualiza parâmetros na direção oposta ao gradiente.
- **SGD (Stochastic Gradient Descent):** variação que usa um único ponto por atualização (mais rápido, pode escapar de mínimos locais).
- **Mini-batch:** subconjunto pequeno de dados usado em cada atualização (compromisso entre SGD e batch GD).
- **Momentum:** técnica que acumula velocidade para atravessar platôs e amortecer oscilações.
- **Hessiana:** matriz de segundas derivadas. Mede a curvatura da superfície de erro.

- **Método de Newton:** usa a Hessiana para convergência mais rápida (mas é caro computacionalmente).
- **Warmup:** aumentar gradualmente a taxa de aprendizado nas primeiras iterações.
- **Regularização:** termo adicionado à perda para penalizar complexidade do modelo (L1, L2).

5.3.2 Mapa Conceitual das Relações



5.3.3 Quando Usar Cada Método?

Situação	Método	Razão
Dataset pequeno ($N < 10^4$)	Batch GD	Cálculo do gradiente completo é viável
Dataset grande ($N > 10^6$)	SGD ou Mini-batch	Batch GD é muito lento
Superfície com platôs	Momentum	Atravessa regiões de baixa curvatura
Modelo convexo (regressão)	Qualquer método converge	Garantias teóricas
Modelo não-convexo (rede neural)	SGD + Momentum + Warmup	Boas práticas empíricas
Modelos massivos (LLMs)	PEFT + AdamW + Warmup	Necessidade de eficiência
Recall de parâmetros exatos	Método de Newton (se viável)	Convergência quadrática
Evitar overfitting	Regularização (L1, L2)	Reduz complexidade

Tabela 5.2: Orientação para escolha do método de otimização.

5.4 Exemplo Didático Passo-a-Passo: Classificação com SGD

5.4.1 Problema: Regressão Logística em 2D

Temos 6 pontos em 2D com classes 0 ou 1. Vamos usar SGD para treinar um classificador linear.

Dados:

```

Ponto:  1  2  3  4  5  6
x1:     1  2  3  8  9  10
x2:     2  2  2  8  8  8
Classe: 0  0  0  1  1  1
  
```

Modelo: regressão logística com pesos w_0 (bias), w_1 , w_2 .

$$\hat{y} = \sigma(w_0 + w_1 x_1 + w_2 x_2) = \frac{1}{1 + e^{-(w_0 + w_1 x_1 + w_2 x_2)}}$$

Perda: entropia cruzada binária $\mathcal{L} = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$

Passo a passo do SGD:

Inicialização: $w_0 = 0, w_1 = 0, w_2 = 0, \eta = 0.1$.

Iteração 1 (amostra 1: $x_1 = 1, x_2 = 2, y = 0$):

$$z = 0 + 0 \cdot 1 + 0 \cdot 2 = 0$$

$$\hat{y} = \sigma(0) = 0.5$$

$$\frac{\partial \mathcal{L}}{\partial w_0} = \hat{y} - y = 0.5 - 0 = 0.5$$

$$\frac{\partial \mathcal{L}}{\partial w_1} = (\hat{y} - y)x_1 = 0.5 \cdot 1 = 0.5$$

$$\frac{\partial \mathcal{L}}{\partial w_2} = (\hat{y} - y)x_2 = 0.5 \cdot 2 = 1.0$$

Atualização:

$$w_0 \leftarrow 0 - 0.1 \cdot 0.5 = -0.05$$

$$w_1 \leftarrow 0 - 0.1 \cdot 0.5 = -0.05$$

$$w_2 \leftarrow 0 - 0.1 \cdot 1.0 = -0.10$$

Iteração 2 (amostra 4: $x_1 = 8, x_2 = 8, y = 1$):

$$z = -0.05 + (-0.05) \cdot 8 + (-0.10) \cdot 8 = -0.05 - 0.40 - 0.80 = -1.25$$

$$\hat{y} = \sigma(-1.25) \approx 0.222$$

$$\frac{\partial \mathcal{L}}{\partial w_0} = \hat{y} - y = 0.222 - 1 = -0.778$$

$$\frac{\partial \mathcal{L}}{\partial w_1} = -0.778 \cdot 8 = -6.224$$

$$\frac{\partial \mathcal{L}}{\partial w_2} = -0.778 \cdot 8 = -6.224$$

Atualização:

$$w_0 \leftarrow -0.05 - 0.1 \cdot (-0.778) = 0.0278$$

$$w_1 \leftarrow -0.05 - 0.1 \cdot (-6.224) = 0.5724$$

$$w_2 \leftarrow -0.10 - 0.1 \cdot (-6.224) = 0.5224$$

Continue iterando... Após várias épocas (passar por todos os pontos várias vezes), os pesos convergem para uma fronteira que separa as classes (Bishop; Duda).

5.5 Aprofundamento: Métodos de Primeira Ordem

5.5.1 Gradiente Descendente: Versões

Seja $\mathcal{L}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N \ell_n(\mathbf{w})$ a função de perda.

1. **Batch Gradient Descent:**

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla \mathcal{L}(\mathbf{w}^{(t)})$$

Usa todos os N pontos. Custo $O(N)$ por iteração (Bishop).

2. **Stochastic Gradient Descent (SGD):**

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla \ell_n(\mathbf{w}^{(t)})$$

Usa um ponto aleatório n por iteração. Custo $O(1)$ por iteração. Pode escapar de mínimos locais devido ao ruído estocástico (Bishop; Goodfellow).

3. **Mini-batch Gradient Descent:**

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \frac{1}{|\mathcal{B}|} \sum_{n \in \mathcal{B}} \nabla \ell_n(\mathbf{w}^{(t)})$$

Usa um subconjunto $\mathcal{B}$ de tamanho B (ex: $B = 32, 64, 128$). Compromisso entre estabilidade e eficiência (Sorvisto).

5.5.2 Momentum e Nesterov Accelerated Gradient

O momentum adiciona um termo de inércia (Duda; Bishop):

$$\begin{aligned} \mathbf{v}^{(t+1)} &= \beta \mathbf{v}^{(t)} + \eta \nabla \mathcal{L}(\mathbf{w}^{(t)}) \\ \mathbf{w}^{(t+1)} &= \mathbf{w}^{(t)} - \mathbf{v}^{(t+1)} \end{aligned}$$

Onde $\beta \in [0, 1)$ é o fator de momentum (tipicamente 0.9). O termo $\mathbf{v}$ acumula gradientes passados, suavizando oscilações.

Nesterov Accelerated Gradient (NAG): calcula o gradiente em uma posição “antecipada” (Goodfellow):

$$\begin{aligned} \mathbf{v}^{(t+1)} &= \beta \mathbf{v}^{(t)} + \eta \nabla \mathcal{L}(\mathbf{w}^{(t)} - \beta \mathbf{v}^{(t)}) \\ \mathbf{w}^{(t+1)} &= \mathbf{w}^{(t)} - \mathbf{v}^{(t+1)} \end{aligned}$$

NAG tem melhores garantias de convergência que momentum padrão.

5.5.3 Otimizadores Modernos: Adam e AdamW

O **Adam** (Adaptive Moment Estimation) combina momentum com adaptação por parâmetro (Goodfellow; Raschka):

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) \nabla \mathcal{L}(\mathbf{w}_{t-1}) \quad (\text{momento de primeira ordem}) \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) (\nabla \mathcal{L}(\mathbf{w}_{t-1}))^2 \quad (\text{momento de segunda ordem}) \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (\text{correção de viés}) \\ \mathbf{w}_t &= \mathbf{w}_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \end{aligned}$$

AdamW é uma variação que desocla o peso da regularização L2 (Raschka), atualmente o otimizador mais comum para LLMs.

5.6 Aprofundamento: Métodos de Segunda Ordem

5.6.1 Método de Newton

O método de Newton usa a Hessiana $\mathbf{H} = \nabla^2 \mathcal{L}(\mathbf{w})$ para incorporar curvatura (Bishop; Duda):

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \mathbf{H}^{-1} \nabla \mathcal{L}(\mathbf{w}^{(t)})$$

5.6.2 Vantagens e Desvantagens

Aspecto	Método de Newton	Gradiente	Descen-
Convergência	Quadrática (muito rápida)	Linear (mais lenta)	dente
Custo por iteração	$O(d^3)$ para inverter a Hessiana	$O(d)$ para calcular gradiente	
Memória	$O(d^2)$ para armazenar Hessiana	$O(d)$ para armazenar gradiente	
Uso prático	Inviável para $d > 10^4$	Padrão para d grande	

Tabela 5.3: Comparação entre métodos de primeira e segunda ordem.

5.6.3 Gradiente Conjugado

O método do gradiente conjugado é um meio-termo: usa informação de gradientes passados para construir direções conjugadas (ortogonais em relação à Hessiana), sem calcular a Hessiana explicitamente (Duda). Converge em no máximo d iterações para problemas quadráticos convexos.

5.7 Aprofundamento: Regularização e Restrições

5.7.1 Regularização L2 (Ridge)

Adiciona o termo $\frac{\lambda}{2} \|\mathbf{w}\|_2^2$ à perda (Bishop; Sorvisto):

$$\mathcal{L}_{\text{ridge}}(\mathbf{w}) = \mathcal{L}_{\text{original}}(\mathbf{w}) + \frac{\lambda}{2} \sum_{j=1}^d w_j^2$$

Efeito: pesos próximos de zero, nenhum peso exatamente zero. A atualização do gradiente fica:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta(\nabla \mathcal{L}_{\text{original}} + \lambda \mathbf{w}^{(t)})$$

5.7.2 Regularização L1 (Lasso)

Adiciona o termo $\lambda \|\mathbf{w}\|_1$ à perda (Bishop; Mitchell):

$$\mathcal{L}_{\text{lasso}}(\mathbf{w}) = \mathcal{L}_{\text{original}}(\mathbf{w}) + \lambda \sum_{j=1}^d |w_j|$$

Efeito: induz esparsidade (muitos pesos exatamente zero). A atualização do gradiente usa subgradi-
entes:

$$\frac{\partial |w_j|}{\partial w_j} = \begin{cases} 1 & \text{se } w_j > 0 \\ -1 & \text{se } w_j < 0 \\ [-1, 1] & \text{se } w_j = 0 \end{cases}$$

5.7.3 Multiplicadores de Lagrange

Para otimização com restrições do tipo $g(\mathbf{w}) = 0$ ou $h(\mathbf{w}) \leq 0$, usamos multiplicadores de Lagrange (Bishop; Duda):

$$\mathcal{L}(\mathbf{w}, \lambda, \mu) = f(\mathbf{w}) + \lambda g(\mathbf{w}) + \mu h(\mathbf{w})$$

Onde λ e μ são os multiplicadores. O ponto ótimo satisfaz $\nabla f + \lambda \nabla g + \mu \nabla h = 0$.

5.8 Análise Formal: Convergência e Taxas de Aprendizado

5.8.1 Convergência para Funções Convexas

Para funções convexas com gradientes L -Lipschitz, o gradiente descendente com taxa de aprendizado $\eta = 1/L$ converge como (Bishop; Goodfellow):

$$\mathcal{L}(\mathbf{w}^{(T)}) - \mathcal{L}(\mathbf{w}^*) \leq \frac{2L\|\mathbf{w}^{(0)} - \mathbf{w}^*\|^2}{T}$$

Ou seja, erro diminui como $O(1/T)$. SGD tem convergência mais lenta: $O(1/\sqrt{T})$, mas o custo por iteração é muito menor.

5.8.2 Convergência para Funções Não-Convexas

Para redes neurais (não-convexas), não há garantias de encontrar o mínimo global. No entanto, em altas dimensões, a maior parte dos mínimos locais tem perda próxima à do mínimo global (Goodfellow). O desafio principal são os **pontos de sela** (onde o gradiente é zero mas não é mínimo), que são mais comuns que mínimos locais.

5.8.3 Escolha da Taxa de Aprendizado

A taxa de aprendizado η é o hiperparâmetro mais crítico (Sorvisto; Raschka):

- η muito grande: divergência (explosão dos gradientes)
- η muito pequeno: convergência extremamente lenta
- η ideal: depende da curvatura da função

Estratégias comuns:

1. **Decaimento exponencial:** $\eta_t = \eta_0 \cdot \gamma^{t/T}$
2. **Decaimento por passo:** $\eta_t = \eta_0 / (1 + \alpha t)$
3. **Cosine annealing:** $\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\frac{\pi t}{T}))$
4. **Warmup:** $\eta_t = \eta_{\max} \cdot \frac{t}{T_{\text{warmup}}}$ para $t < T_{\text{warmup}}$ (Raschka)

5.9 Comparação Entre Otimizadores

Otimizador	Velocidade	Memória	Hiperparâmetros	Quando usar
Batch GD	Lenta	Alta ($O(Nd)$)	η	Dataset pequeno
SGD	Média	Baixa ($O(d)$)	η	Dataset grande, convexo
SGD + Momentum	Média	Baixa	η, β	Superfície com platôs
Adam	Rápida	Média ($O(2d)$)	η, β_1, β_2	Redes neurais, NLP
AdamW	Rápida	Média	$\eta, \beta_1, \beta_2, \lambda$	LLMs, imagem
Método de Newton	Muito rápida	Alta ($O(d^2)$)	Nenhum	d pequeno, convexo

Tabela 5.4: Comparação entre otimizadores. Fonte: Bishop; Raschka; Goodfellow

5.10 Limitações e Quando a Otimização Falha

5.10.1 Custos e Desvantagens

1. **Gradientes evanescentes (vanishing gradients):** em redes profundas, gradientes das camadas iniciais podem se tornar exponencialmente pequenos (Goodfellow). Resolvido com ReLU, batch norm, resnets.
2. **Gradientes explosivos (exploding gradients):** gradientes podem crescer exponencialmente (especialmente em RNNs). Resolvido com gradient clipping.
3. **Pontos de sela:** em altas dimensões, são mais comuns que mínimos locais. SGD com momentum consegue escapar.
4. **Mínimos locais ruins:** raros em altas dimensões, mas podem ocorrer.

5.10.2 Quando a Otimização Não Ajuda

- **Função de perda mal especificada:** se a perda não reflete o objetivo real, otimizar é inútil.
- **Dados não representativos:** otimizar o modelo para minimizar perda em dados enviesados não generaliza.
- **Overfitting:** mesmo que a otimização funcione perfeitamente (perda zero no treino), o modelo pode não generalizar.
- **Problemas NP-difíceis:** alguns problemas de otimização combinatorial são intratáveis.

5.11 Pesquisa Atual e Fronteiras

5.11.1 Otimização para LLMs

Treinar LLMs apresenta desafios únicos (Raschka; Alammari):

- **AdamW:** padrão para LLMs (combina Adam com decaimento de peso correto).
- **Warmup linear + decaimento cosseno:** estabiliza o treinamento no início.
- **Gradient clipping:** previne explosão de gradientes (valores típicos: 1.0)
- **Bfloat16:** precisão mista reduz memória pela metade.
- **Zero Redundancy Optimizer (ZeRO):** distribui parâmetros, gradientes e momentos entre GPUs.

5.11.2 Otimização para PEFT (LoRA)

O LoRA (abordado no capítulo de Transferência) reduz drasticamente o número de parâmetros a otimizar (Raschka). A otimização pode ser feita com:

- AdamW (padrão)
- Taxa de aprendizado mais alta ($\eta = 10^{-3}$ a 10^{-4})
- Poucas épocas (2 a 5)

5.11.3 Otimização Neuronal e Meta-Aprendizado

Pesquisas atuais exploram **otimizadores aprendidos** (meta-aprendizado): uma rede neural aprende a otimizar outra rede (Goodfellow). Exemplo: *LSTM optimizer*. Ainda experimental, mas promissor para otimização de hiperparâmetros.

5.11.4 Otimização Distribuída

Com o crescimento dos modelos, a otimização distribuída se tornou essencial (Sorvisto):

- **Sincronizada (All-Reduce):** todos os workers compartilham gradientes a cada passo. Mais estável, mas comunicação cara.
- **Assíncrona (Parameter Server):** workers atualizam parâmetros de forma independente. Mais rápida, mas pode levar a instabilidade.

5.11.5 Otimização Quântica

Embora ainda distante, há pesquisas em algoritmos de otimização quântica (ex: QAOA) que, se viabilizados, poderiam acelerar drasticamente o treinamento de certas classes de modelos.

5.12 Resumo e Conexões com Outros Capítulos

5.12.1 O que Você Deve Lembrar

1. **Ideia central:** minimizar a perda ajustando parâmetros na direção do gradiente negativo.
2. **GD vs. SGD vs. Mini-batch:** trade-off entre estabilidade, velocidade e uso de memória.
3. **Momentum:** ajuda a atravessar platôs e amortece oscilações.
4. **Adam/AdamW:** otimizadores adaptativos padrão para redes neurais e LLMs.
5. **Regularização L1/L2:** penaliza complexidade do modelo para evitar overfitting.
6. **Taxa de aprendizado:** hiperparâmetro mais crítico (warmup + decaimento).

5.12.2 Conexões com Outros Capítulos

Este capítulo sobre Otimização de Modelos se conecta com os demais capítulos do resumo das seguintes formas:

- **Ensemble Learning (Capítulo 1):** O AdaBoost minimiza sequencialmente uma função de erro exponencial, que é um problema de otimização. O gradiente descendente (e suas variações) é usado para treinar cada classificador fraco do ensemble. A compreensão de otimização ajuda a entender por que o AdaBoost converge e como escolher a taxa de aprendizado.
- **Aprendizado Não-Supervisionado (Capítulo 2):** O algoritmo *Expectation-Maximization* (EM) — usado para treinar modelos de mistura Gaussianas (GMMs) — é um método de otimização para variáveis latentes. A decomposição em passo E e passo M é análoga a coordenada descendente. O K-means (Seção 5 do Capítulo 2) é um caso extremo do EM.
- **Aprendizado Semi-Supervisionado (Capítulo 3):** O EM com rótulos parciais (Seção 5 do Capítulo 3) é uma aplicação direta de otimização em SSL. As mesmas técnicas de otimização (gradiente, Newton, Adam) usadas no Capítulo 5 são aplicadas no treinamento de classificadores dentro do loop de auto-treinamento e aprendizado ativo.
- **Aprendizado por Transferência (Capítulo 4):** O fine-tuning (Seção 5 do Capítulo 4) é um problema de otimização onde o ponto inicial é o modelo pré-treinado (em vez de pesos aleatórios). Técnicas como taxa de aprendizado diferenciada (menor para camadas iniciais) e regularização $\|\mathbf{w} - \mathbf{w}_0\|^2$ conectam diretamente os fundamentos de otimização com a prática de transferência.

Resumo das relações:

Capítulo	Conexão com Otimização
Ensemble (1)	AdaBoost como otimização exponencial
Não-Supervisionado (2)	EM para GMMs e K-means
Semi-Supervisionado (3)	EM com rótulos parciais
Transferência (4)	Fine-tuning como otimização com inicialização especial

5.12.3 Cheat Sheet para Revisão Rápida (Leitores Experientes)

Quero...	Uso...	Fórmula chave
Otimizar função convexa	Batch GD	$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla \mathcal{L}$
Processar dataset gigante	SGD ou Mini-batch	$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla \ell_n$
Atravessar platôs	Momentum	$\mathbf{v} \leftarrow \beta \mathbf{v} + \eta \nabla \mathcal{L}$
Otimizar rede neural	Adam / AdamW	$\mathbf{w}_t = \mathbf{w}_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$
Penalizar pesos grandes	Regularização L2	$\mathcal{L}_{\text{ridge}} = \mathcal{L}_{\text{original}} + \frac{\lambda}{2} \ \mathbf{w}\ ^2$
Induzir esparsidade	Regularização L1	$\mathcal{L}_{\text{lasso}} = \mathcal{L}_{\text{original}} + \lambda \ \mathbf{w}\ _1$
Prevenir overfitting	Early stopping, dropout	Validar em conjunto de validação

Tabela 5.5: Resumo rápido para consulta.